from Hadoop Parquet Files to SQL

HiveMetaStore & Partitions

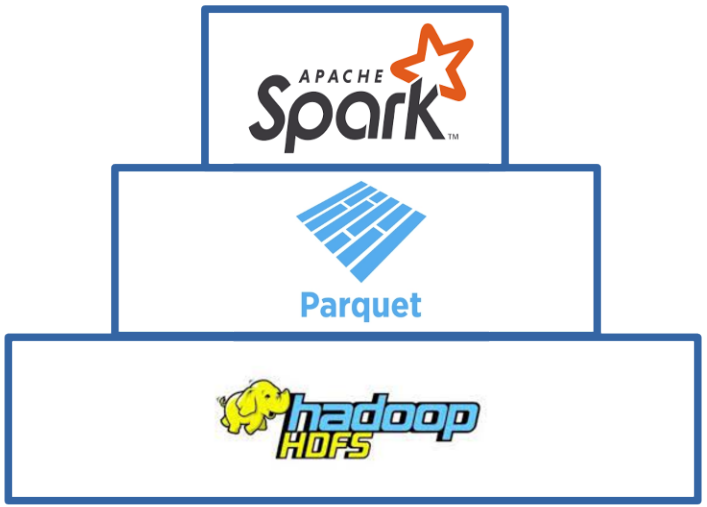
Course Esilv 2026
arnaud.nauwynck@gmail.com

this document:
https://github.com/Arnaud-Nauwynck/presentations/tree/main/pres-bigdata
7-Sql-to-Hadoop-files-parquet-metastore

Outline

- From Hadoop path + Parquet Files to Table
- Datalake Architectures
  From Hive MetaStore to Iceberg Rest Catalog
- DDL
- DML - read/write Concurrency problems
- Partitioning
- Partition Pruning Optim
- Iceberg Versionning

Hadoop FileSystem + Parquet Files




```
val baseStorageLocation =
“s3://bucket/datalake/team/dir/subDir”;

val dataset =
    spark
      .read
      .parquet(baseStorageLocation + “/dataset/part=*/”)
```

DataSet API … want High Level SQL

Low level: “Dataset” = */*.parquet files

High level view: “table”

Hard-coded path: s3://.../part=*/

Config mapping : “table name” => path

Complex Dataset API / scala code

SQL





DataSet.createOrReplaceTempView → table for SELECT SQL

```
scala> val ds = spark.read.parquet("C:/data/OpenData-gouv.fr/new-parquet3-sorted")
val ds: org.apache.spark.sql.DataFrame = [uid_adresse: string, cle_interop: string ... 17 more fields]

scala> ds.select("voie_nom").distinct().count()
val res0: Long = 1305099

scala> ds.createOrReplaceTempView("addr")

scala> spark.sql("select count(distinct voie_nom) from addr").show()
+-----------------------+
|count(DISTINCT voie_nom)|
+-----------------------+
|                1305098|
+-----------------------+
```

OK - can do SQL over DataSet



But SELECT (read) Only



Dataset.write.saveAsTable(“addr”)

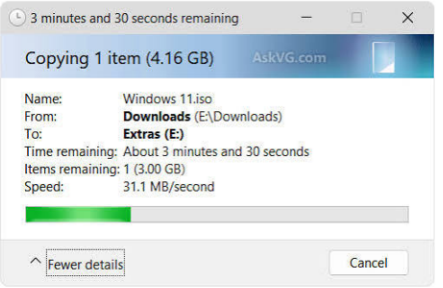
```
scala> ds.write.saveAsTable("addr")
[Stage 13:==========================>                              (8 + 8) / 17]
```

Create table ok



… But copying data !
(to managed location)





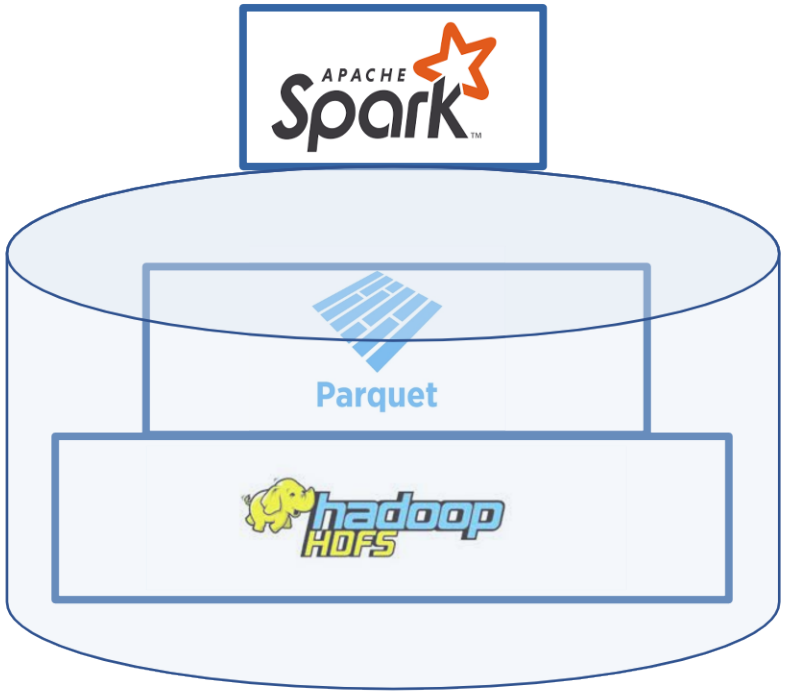
Dataset ds = spark.sql(“ … “)

```
scala> spark.sql("select count(distinct voie_nom) from addr").show()
+------------------------+
|count(DISTINCT voie_nom)|
+------------------------+
|                 1305098|
+------------------------+
```

```
scala> val ds2 = spark.sql("select count(distinct voie_nom) from addr")
val ds2: org.apache.spark.sql.DataFrame = [count(DISTINCT voie_nom): bigint]

scala> ds2.show()
[Stage 16:=========================================>                (12 + 5) / 17]
```

Table over Dir / Parquet Files



val ds = spark.sql(“ SELECT * FROM **table** ”)

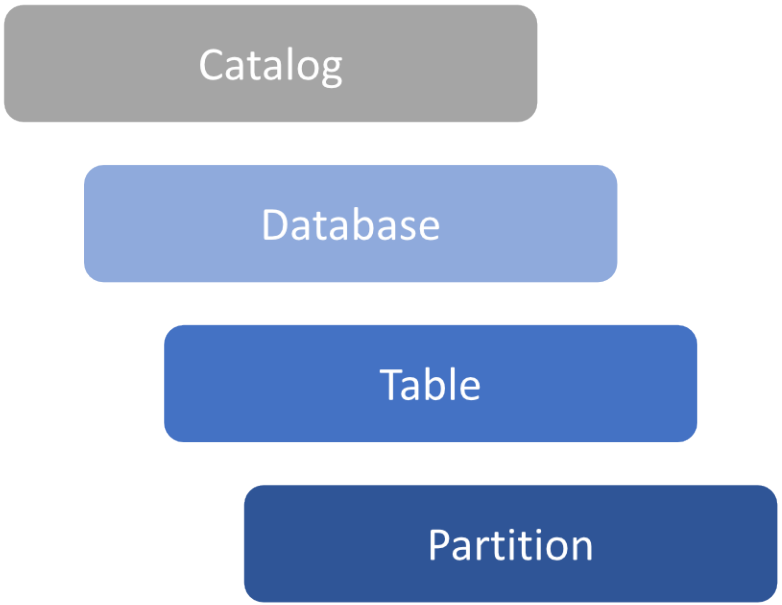
Table ⇒ hide storage details

Mapping (= metadata)

Table name in SQL ✯ location dir

Listing .../**/*.parquet files

Metastore Concepts



= network connection to MetaStore server

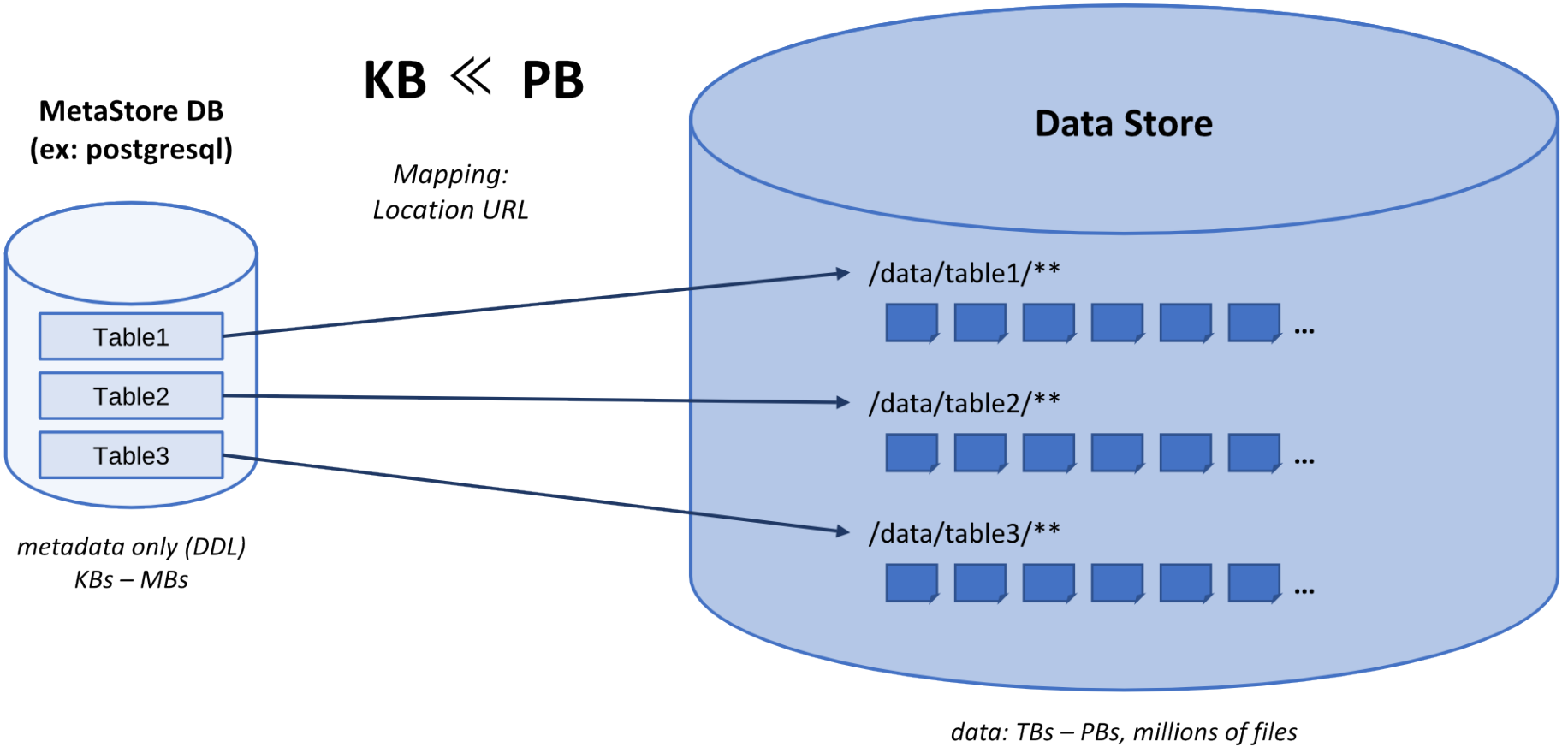
= namespace per team (read/write permission)

= schema + location

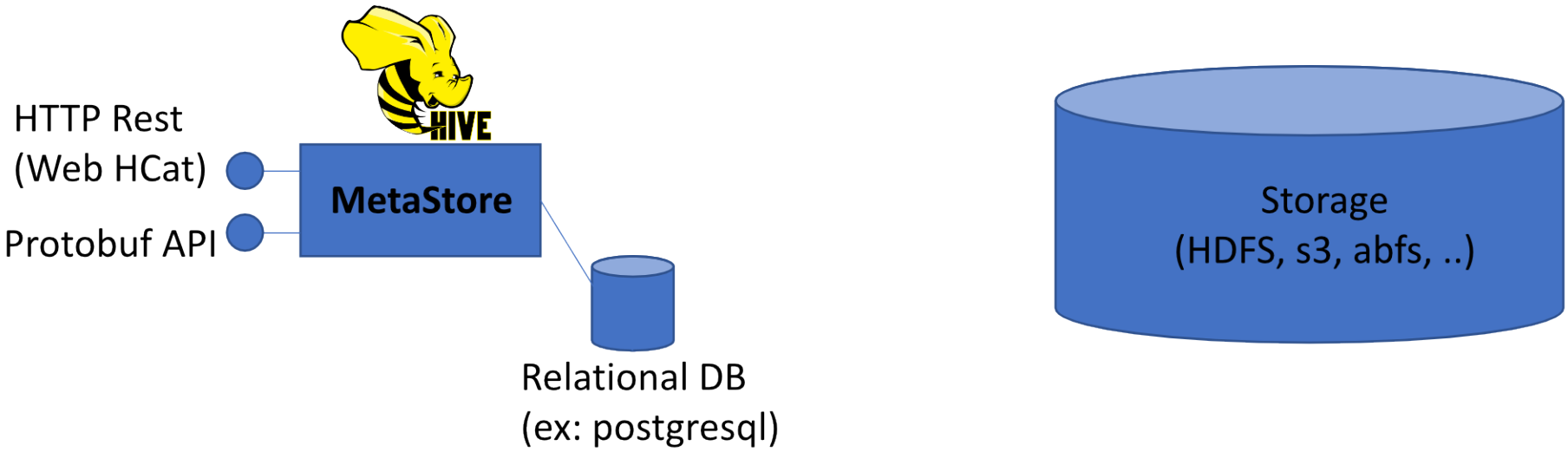
Outline

- From Hadoop path + Parquet Files to Table
- Datalake Architectures
  From Hive MetaStore to Iceberg Rest Catalog
- DDL
- DML - read/write Concurrency problems
- Partitioning
- Partition Pruning Optim
- Iceberg Versionning

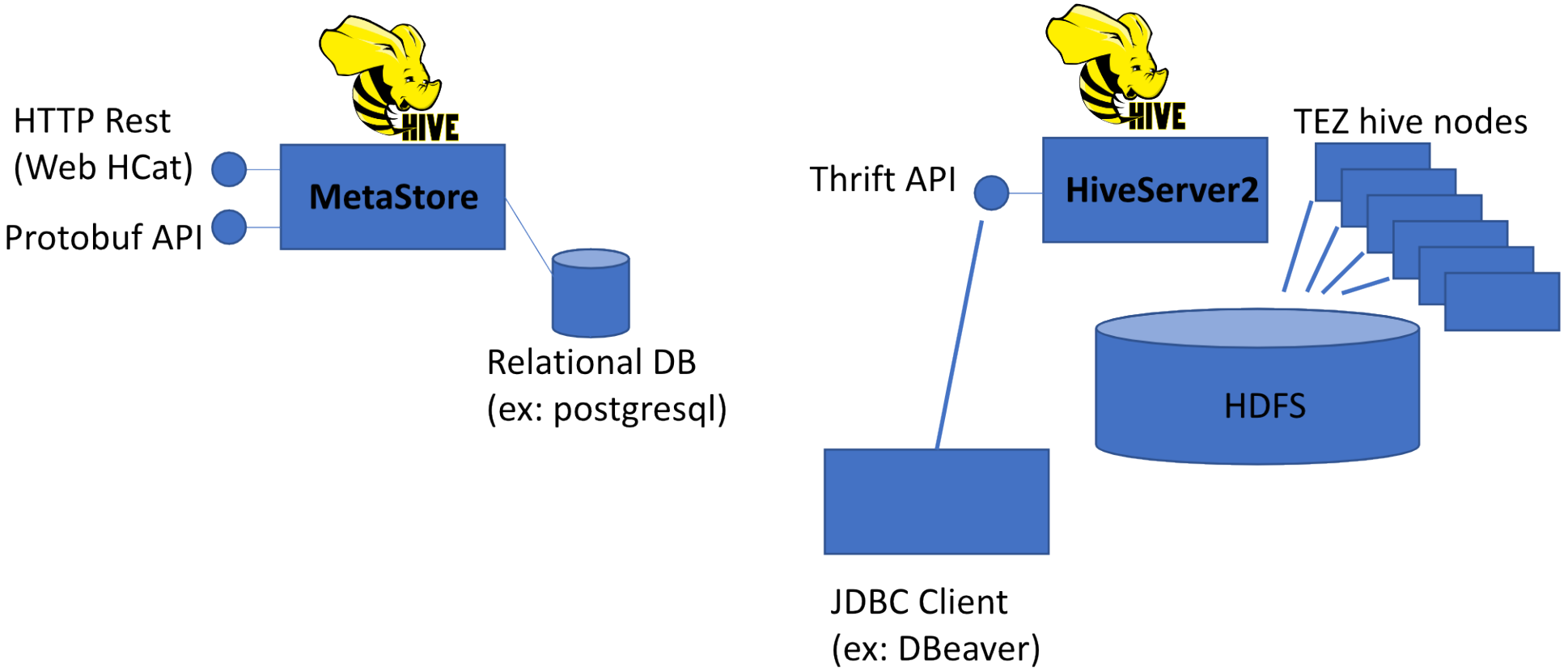
Catalog MetaStore



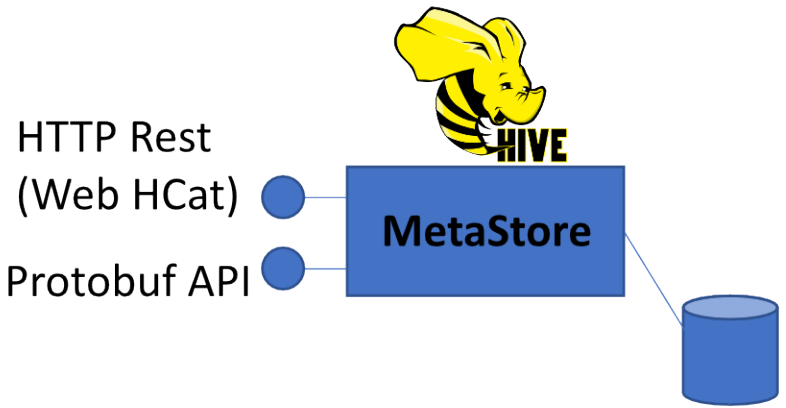
Hive MetaStore Architecture

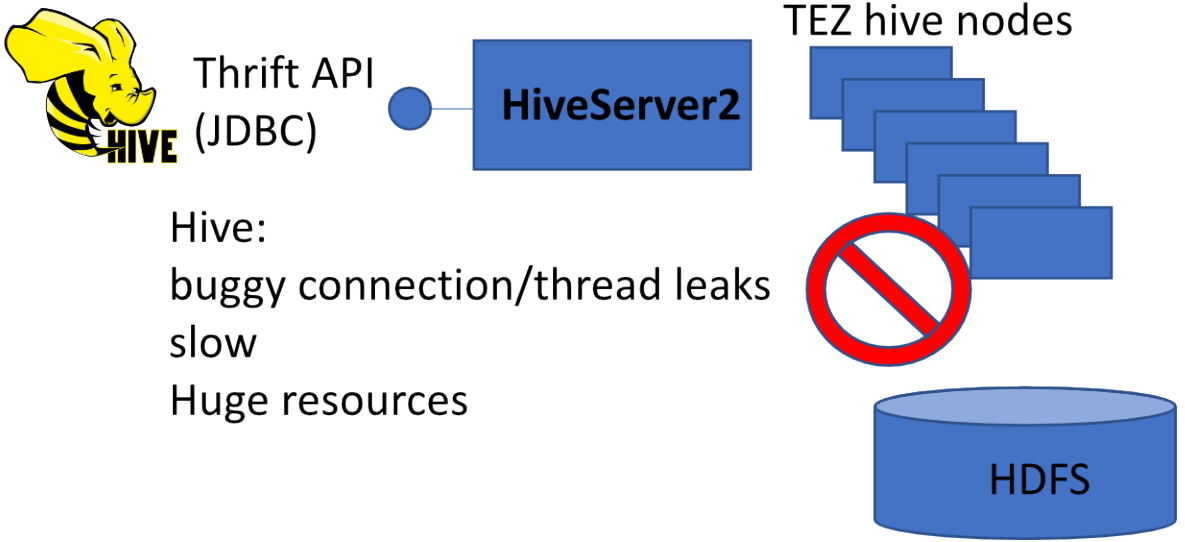


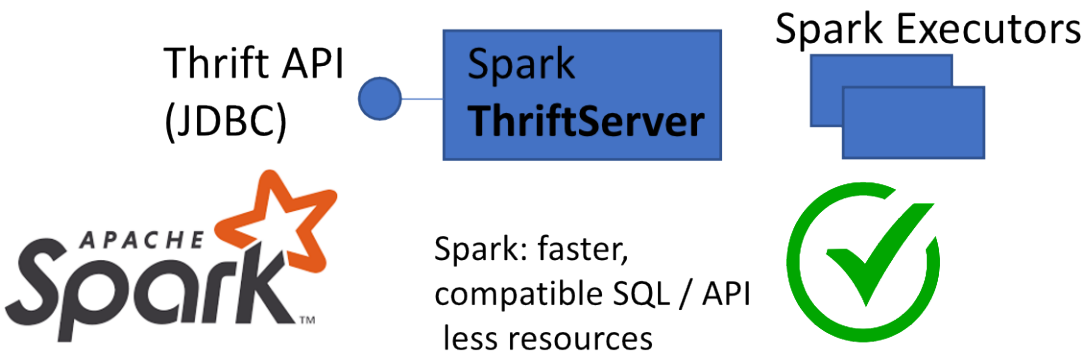
Hive MetaStore != Hive Server2



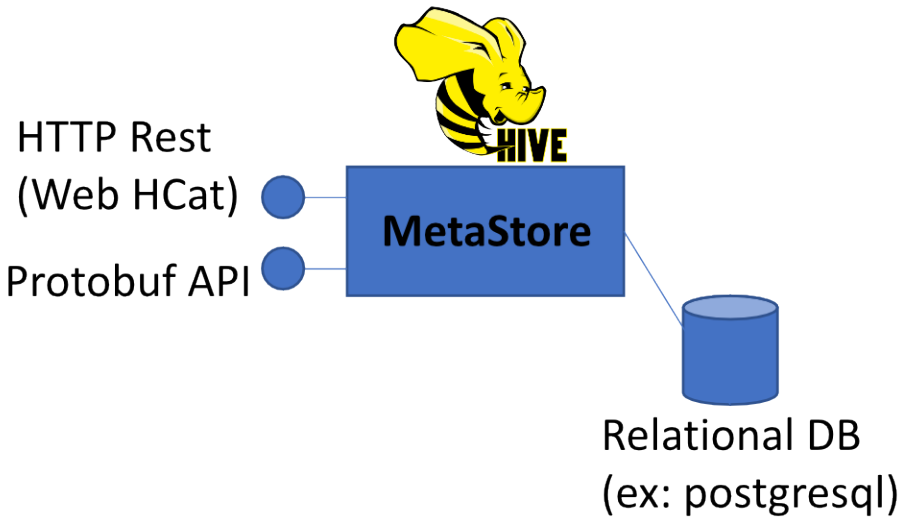
Hive Server2 ... Deprecated







Many SQL Engines / 1 MetaStore



APACHE Spark™

databricks *photon*

HIVE

trino

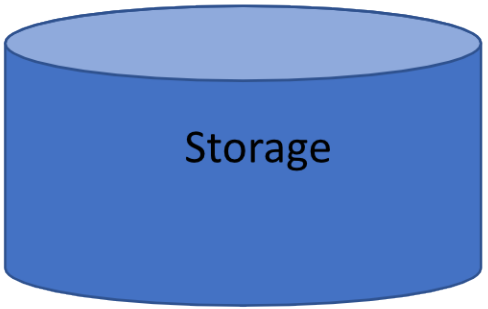
ClickHouse

APACHE DATAFUSION™

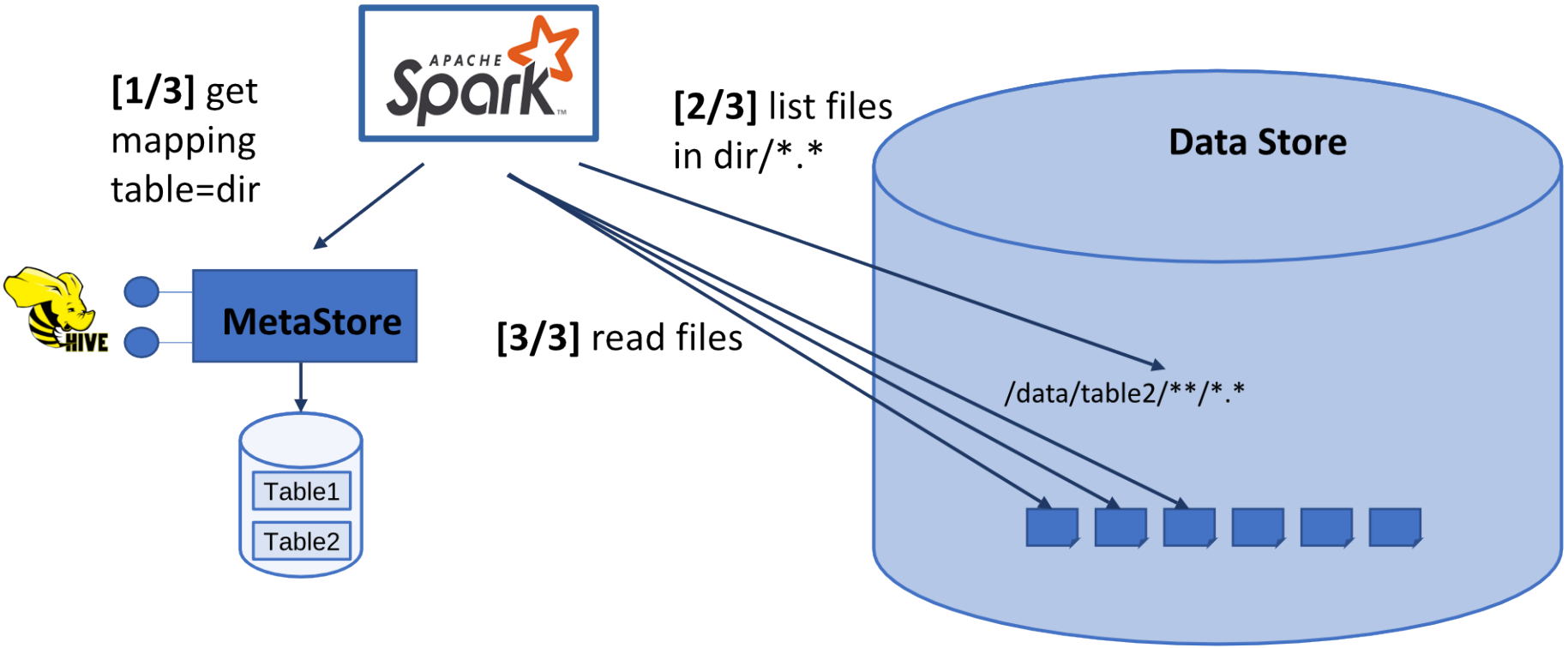
Velox

BigQuery

Microsoft Fabric



1 query MetaStore (Mapping)
⇒ 2 Directory Listing
⇒ 3 Data Read



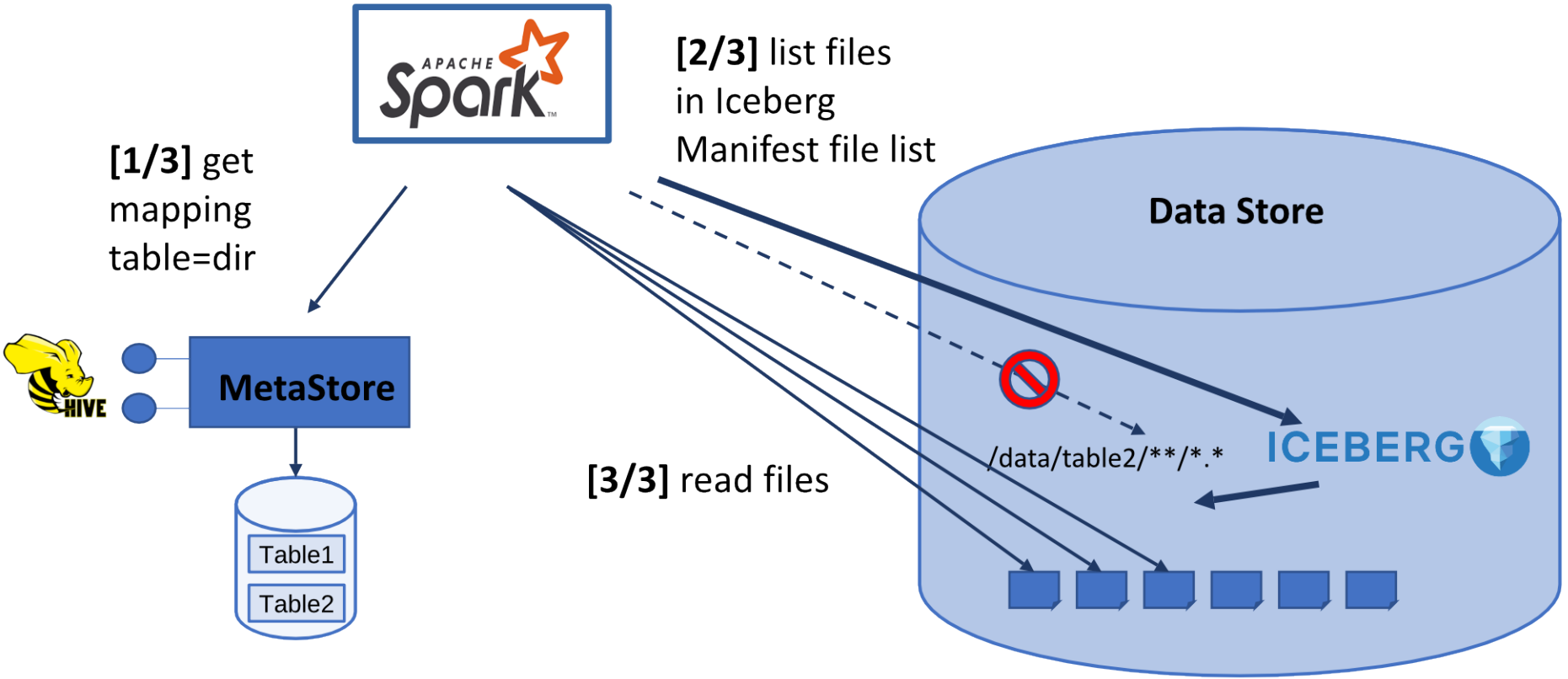
Dir Listing / Renaming / … Problems

On Aws S3, listing is slow,
renaming is not atomic

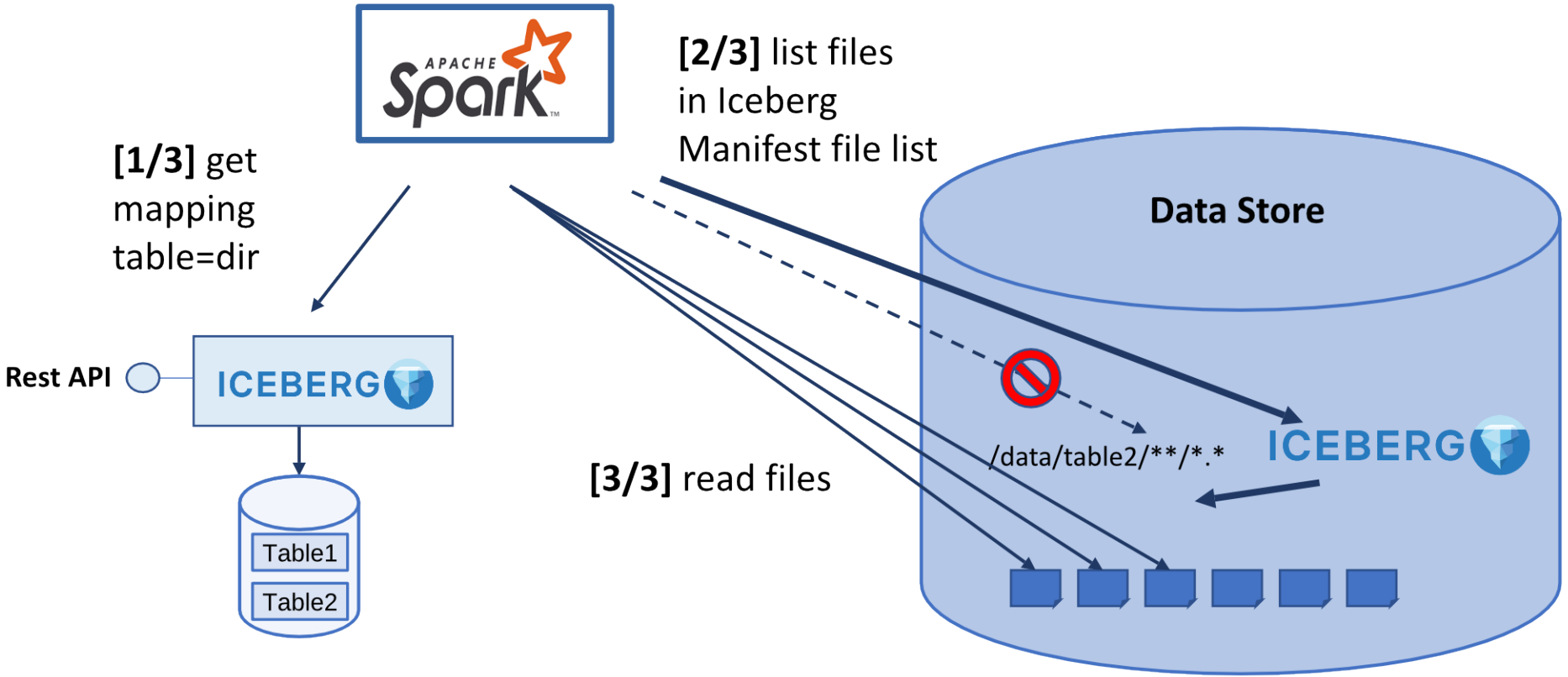
Having >> thousands or files per dir cause performance issues, Timeouts errors, ...

Having >> millions of dirs cause Hadoop HDFS issues

Spoiler Alert … Iceberg indirection to avoid Directory Listing



Spoiler Alert 2 … Iceberg Rest Catalog replacing Hive MetaStore



Many IRC (Iceberg Rest Catalog) Implementations



https://polaris.apache.org/



https://github.com/lakekeeper/



https://gravitino.apache.org/



https://www.databricks.com/product/unity-catalog
https://github.com/unitycatalog



https://projectnessie.org/



https://aws.amazon.com/glue/

Outline

- From Hadoop path + Parquet Files to Table
- Datalake Architectures
  From Hive MetaStore to Iceberg Rest Catalog
- DDL
- DML - read/write Concurrency problems
- Partitioning
- Partition Pruning Optim
- Iceberg Versionning

DDL

DDL : Data **Definition** Langage

(!= DML : Data **Modification** Langage)

Example DDL - DML

DDL

Show databases
Create database {db}
Drop|Delete database {db}

Show tables
Create table {db.table} ({cols}...)...
Drop|Delete table {db.table}
Alter Table ...

Show partitions {db.table}
Alter Table … add partition ...

Show functions
...

DML

SELECT … FROM {db.table} WHERE...

INSERT (INTO) {db.table}
TRUNCATE TABLE {db.table}

(spark extension with Iceberg)
UPDATE {db.table} SET … WHERE ...
UPSERT ...
DELETE ...

spark.sql(“ Show Tables “).show()

```
scala> spark.sql("show tables").show()
+---------+---------+-----------+
|namespace|tableName|isTemporary|
+---------+---------+-----------+
|  default|     addr|      false|
+---------+---------+-----------+
```

spark.catalog

```
scala> spark.catalog.list
listCatalogs(     listDatabases(    listPartitions(   listViews(
listColumns(      listFunctions(    listTables(
```

```
scala> spark.catalog.listTables().show()
+-----+-------------+---------+-----------+---------+-----------+
| name|      catalog|namespace|description|tableType|isTemporary|
+-----+-------------+---------+-----------+---------+-----------+
| addr|spark_catalog|[default]|       NULL|  MANAGED|      false|
|addr2|spark_catalog|[default]|       NULL| EXTERNAL|      false|
+-----+-------------+---------+-----------+---------+-----------+
```

Show Create Table

```
scala> spark.sql("show create table addr").foreach(println(_))
[CREATE TABLE spark_catalog.default.addr (
  uid_adresse STRING COLLATE UTF8_BINARY,
  cle_interop STRING COLLATE UTF8_BINARY,
  commune_insee STRING COLLATE UTF8_BINARY,
  commune_nom STRING COLLATE UTF8_BINARY,
  commune_deleguee_insee STRING COLLATE UTF8_BINARY,
  commune_deleguee_nom STRING COLLATE UTF8_BINARY,
  voie_nom STRING COLLATE UTF8_BINARY,
  lieudit_complement_nom STRING COLLATE UTF8_BINARY,
  numero INT,
  suffixe STRING COLLATE UTF8_BINARY,
  position STRING COLLATE UTF8_BINARY,
  x DOUBLE,
  y DOUBLE,
  long DOUBLE,
  lat DOUBLE,
  cad_parcelles STRING COLLATE UTF8_BINARY,
  source STRING COLLATE UTF8_BINARY,
  date_der_maj DATE,
  certification_commune STRING COLLATE UTF8_BINARY)
USING parquet
]
```

Describe table extended

```
scala> spark.sql("describe table extended addr").show(100, false)
+----------------------------+----------------------------------------------------------------+-------+
|col_name                    |data_type                                                       |comment|
+----------------------------+----------------------------------------------------------------+-------+
|uid_adresse                 |string                                                          |NULL   |
|cle_interop                 |string                                                          |NULL   |
|commune_insee               |string                                                          |NULL   |
|commune_nom                 |string                                                          |NULL   |
|                            |                                                                |       |
|# Detailed Table Information|                                                                |       |
|Catalog                     |spark_catalog                                                   |       |
|Database                    |default                                                         |       |
|Table                       |addr                                                            |       |
|Owner                       |arnaud                                                          |       |
|Created Time                |Thu Oct 22 18:48:15 CEST 2026                                   |       |
|Last Access                 |UNKNOWN                                                         |       |
|Created By                  |Spark 4.2.0                                                     |       |
|Type                        |MANAGED                                                         |       |
|Provider                    |parquet                                                         |       |
|Location                    |file:/C:/apps/spark/spark-warehouse/addr                        |       |
|Serde Library               |org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe     |       |
|InputFormat                 |org.apache.hadoop.hive.ql.io.parquet.MapredParquetInputFormat   |       |
|OutputFormat                |org.apache.hadoop.hive.ql.io.parquet.MapredParquetOutputFormat  |       |
+----------------------------+----------------------------------------------------------------+-------+
```

Drop Table

```
scala> spark.sql("DROP TABLE addr2").show()
++
||
++
++
```

Drop MANAGED Table
⇒ Delete DATA



Drop (MANAGED) Table + Re-create

… Lose all Data !!

Create EXTERNAL Table … Location

```
scala> spark.sql("""
     |  CREATE EXTERNAL TABLE addr2 (
     |    uid_adresse STRING COLLATE UTF8_BINARY,
     |    cle_interop STRING COLLATE UTF8_BINARY,
     |    commune_insee STRING COLLATE UTF8_BINARY,
     |    commune_nom STRING COLLATE UTF8_BINARY,
     |    commune_deleguee_insee STRING COLLATE UTF8_BINARY,
     |    commune_deleguee_nom STRING COLLATE UTF8_BINARY,
     |    voie_nom STRING COLLATE UTF8_BINARY,
     |    lieudit_complement_nom STRING COLLATE UTF8_BINARY,
     |    numero INT,
     |    suffixe STRING COLLATE UTF8_BINARY,
     |    position STRING COLLATE UTF8_BINARY,
     |    x DOUBLE,
     |    y DOUBLE,
     |    long DOUBLE,
     |    lat DOUBLE,
     |    cad_parcelles STRING COLLATE UTF8_BINARY,
     |    source STRING COLLATE UTF8_BINARY,
     |    date_der_maj DATE,
     |    certification_commune STRING COLLATE UTF8_BINARY)
     |  USING parquet
     |  LOCATION "file:/C:/data/addr2"
     |  """).show()
++
||
++
++
```



Drop EXTERNAL Table
⇒ metadata only



Drop (EXTERNAL) Table + Re-create

… OK !!

Managed / External Table

```
create table XX (
   col1 type1, col2 ...
)
```

Managed Table

DROP ⇒ deletes metadata AND data

```
create EXTERNAL table XX (
   col1 type1, col2 ...
)
LOCATION
's3://datalake/dir/..../table'
```

External Table

DROP ⇒ deletes metadata only
Data exists independently of any metastore

MetaStore = Location + Schema + Format + Partition ...

Location: name in SQL ⇔ dir path in storage

Schema : column types

Format: file encoding (parquet, iceberg, vortex, csv, json, …)

Partitions: group by sub-directory /col={value}/*.*

Reminder spark read Schema

Each parquet file contains its own schema

Spark can
A/ choose “randomly” a single file for the schema (?)

or
B/ use merge schemas
spark.read.**option(“mergeSchema”, true)**
or
C/ read with a “coerced” schema:
spark.read**.schema(“ col1 type1, col2 type2, ...”)**

Table Schema in MetaStore = coerce read schema

Fast

Deterministe

Support file evolution (add columns ...)

Sample CREATE EXTERNAL TABLE

```
CREATE EXTERNAL TABLE db.student (
   id int,
   firstName string,
   lastName string
)
PARTITIONED BY (
  promo int
)
STORED AS parquet
LOCATION ‘/data/student’
```

Supports Structured Data (Map,List,Struct)

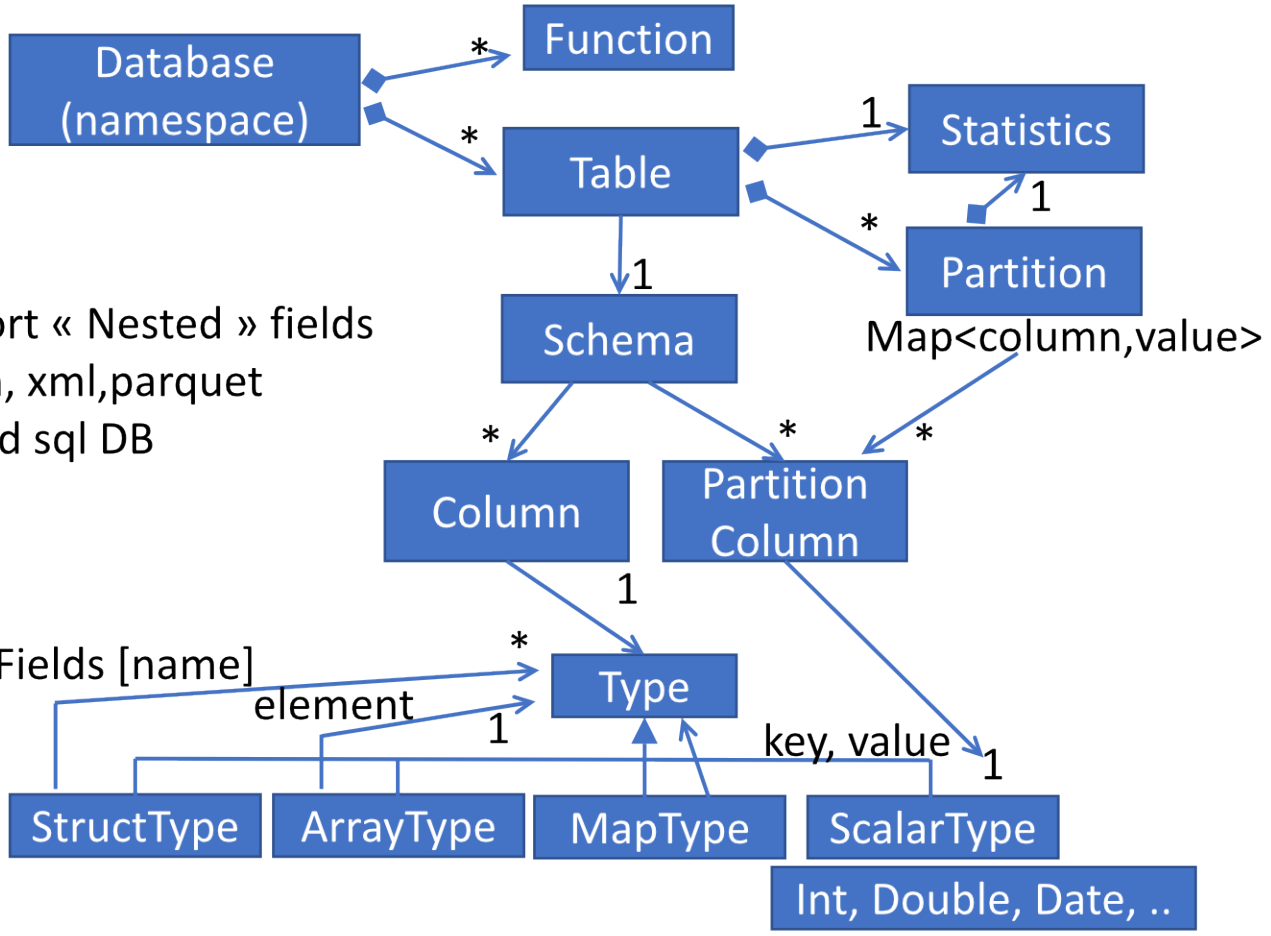
classical SQL database support only simple flat column types
⇒ Use Normal Forms + many sql JOIN between tables

!= bigdata engines,
⇒ use nested denormalized (pre-join) columns

Advanced CREATE EXTERNAL TABLE

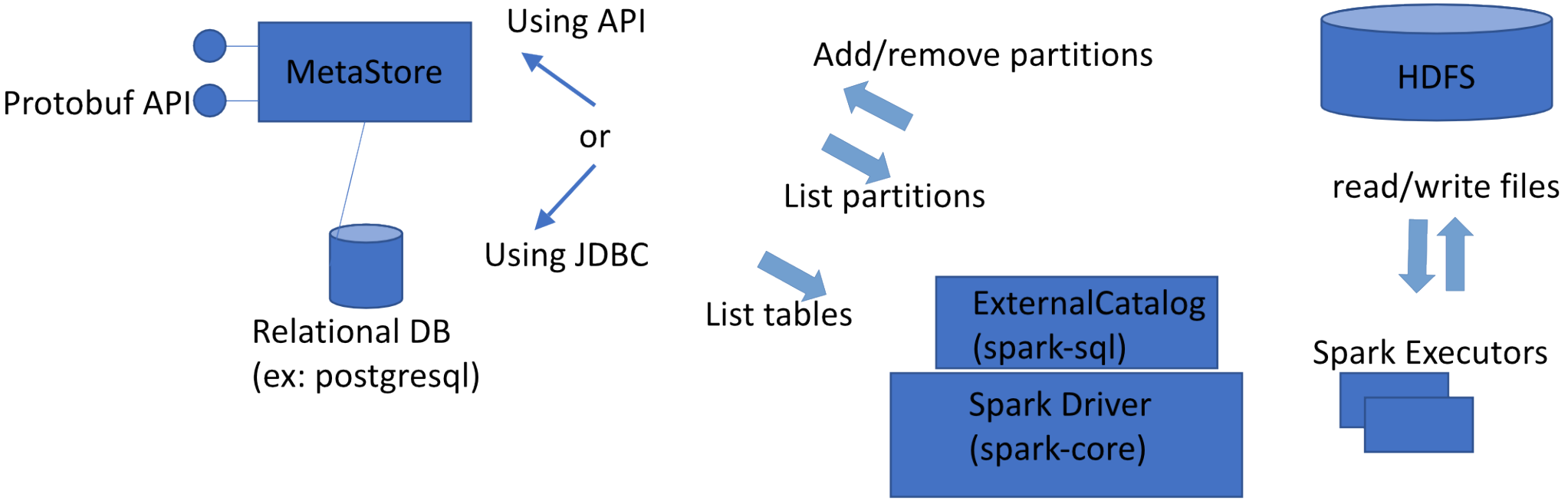
```
CREATE EXTERNAL TABLE db.student (
  id int, firstName string, lastName string,
  address struct< street string,number int,zipcode int >,
  graduations array< struct< name string, obtentionDate date > >,
  extraData map< string,string >
)
PARTITIONED BY ( promo int )
CLUSTERED BY ( id, ...)  SORTED BY (lastName, firstName )
STORED AS parquet
LOCATION '/data/student'
```

MetaStore Model



Schema support « Nested » fields
like typed json, xml,parquet
unlike standard sql DB

Spark supports Hive MetaStore



Outline

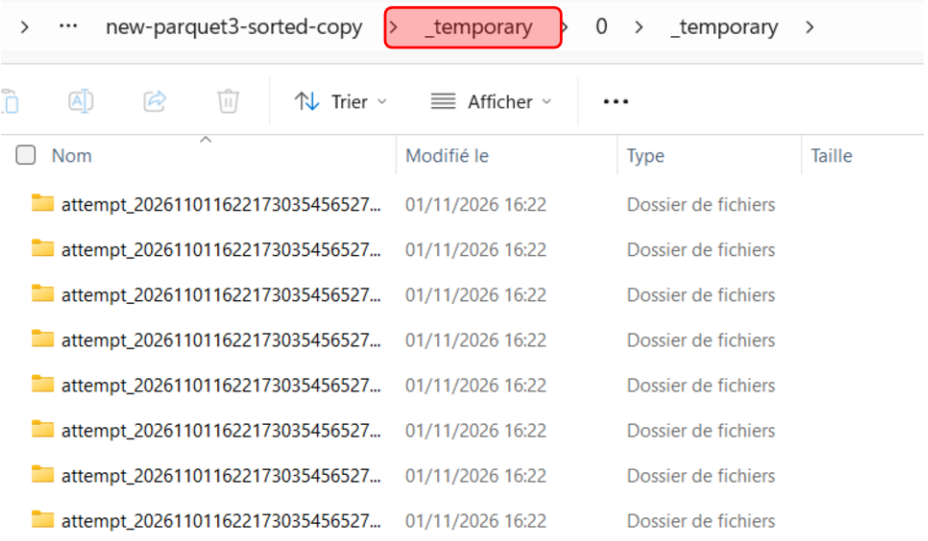
- From Hadoop path + Parquet Files to Table
- Datalake Architectures
  From Hive MetaStore to Iceberg Rest Catalog
- DDL
- DML - read/write Concurrency problems
- Partitioning
- Partition Pruning Optim
- Iceberg Versionning

Write ⇒ to _temporary/** then rename

Spark try to be as much as possible “ACID”:

Isolated : by writing to “_temporary/**”, which is hidden directory, so skipped for any others

Atomic: by doing “rename” Dirs or Files on commit



Write … NOT really « ACID »


Atomic

C onsistent

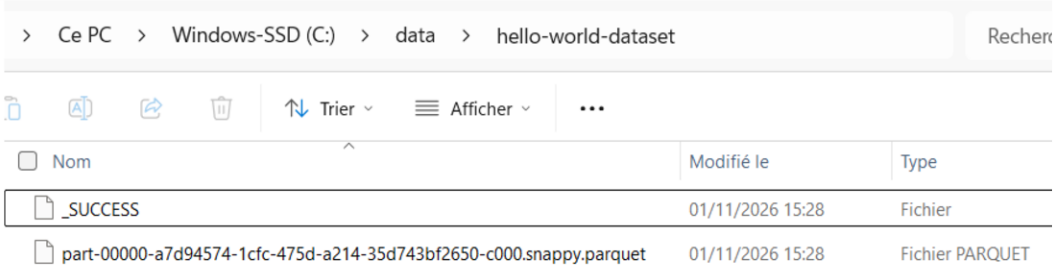

Isolated

D urable

By Default, spark write in empty (non-existent) dir only

```
scala> val ds = spark.createDataset( Seq("Hello", "World") )
val ds: org.apache.spark.sql.Dataset[String] = [value: string]

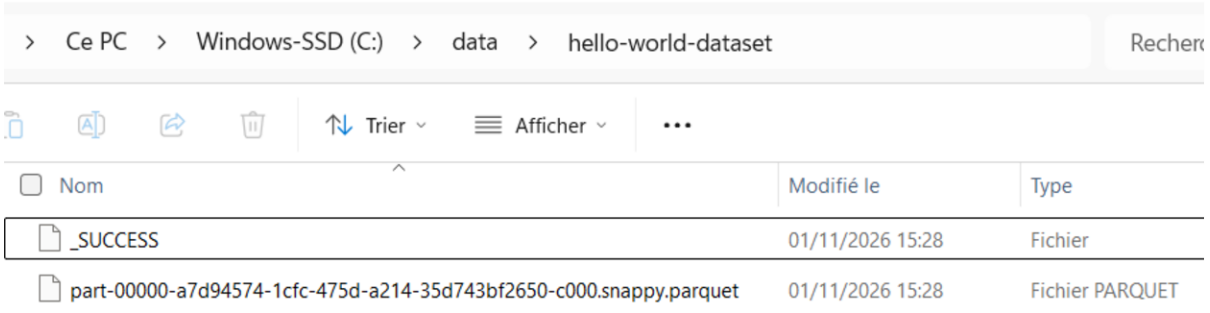
scala> ds.repartition(1).write.mode("overwrite").save("c:/data/hello-world-dataset")
```



Retry write ⇒ Error : **Path already exist … set mode as “overwrite” to overwrite existing path**

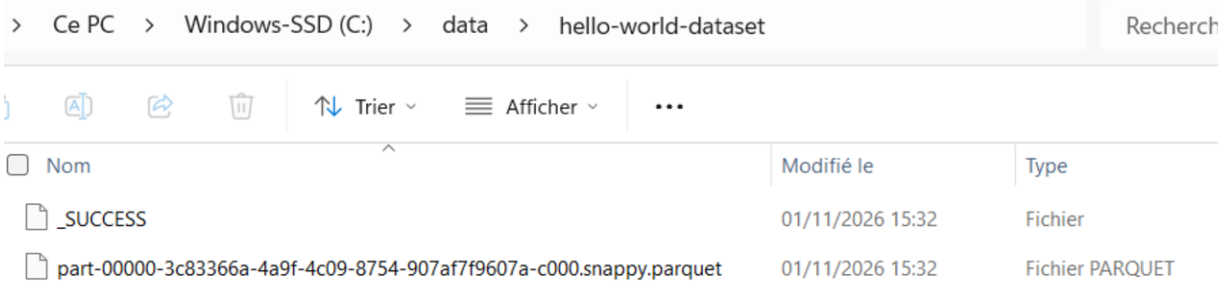
```
scala> ds.repartition(1).write.save("c:/data/hello-world-dataset1")
org.apache.spark.sql.AnalysisException: [PATH_ALREADY_EXISTS] Path file:/c:/data/hello-world-dataset1 already exists. Se
t mode as "overwrite" to overwrite the existing path. SQLSTATE: 42K04
  at org.apache.spark.sql.errors.QueryCompilationErrors$.outputPathAlreadyExistsError(QueryCompilationErrors.scala:1971)
  at org.apache.spark.sql.execution.datasources.InsertIntoHadoopFsRelationCommand.run(InsertIntoHadoopFsRelationCommand.
```

dataset.write.mode(“overwrite”)




```
scala> ds.repartition(1).write.mode("overwrite").save("c:/data/hello-world-dataset")
```

NEW



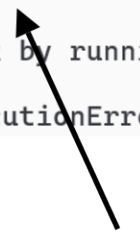
Re-read after externally modified
⇒ ERROR File Not Exist

```
scala> val ds = spark.read.load("c:/data/hello-world-dataset")
val ds: org.apache.spark.sql.DataFrame = [value: string]

scala> ds.count()
val res0: Long = 2

scala> // ... wait few mn, file externally changed

scala> ds.count()
26/11/01 15:52:37 ERROR Executor: Exception in task 0.0 in stage 4.0 (TID 3)
org.apache.spark.SparkException: [FAILED_READ_FILE.FILE_NOT_EXIST] Encountered error while reading file file:///c:/data/
hello-world-dataset/part-00000-3c83366a-4a9f-4c09-8754-907af7f9607a-c000.snappy.parquet. File does not exist. It is poss
ible the underlying files have been updated.
You can explicitly invalidate the cache in Spark by running 'REFRESH TABLE tableName' command in SQL or by recreating th
e Dataset/DataFrame involved. SQLSTATE: KD001
        at org.apache.spark.sql.errors.QueryExecutionErrors$.fileNotExistError(QueryExecutionErrors.scala:902)
```



Dataset now points to “old” Files
(= file names discovered during first spark.read.load(“...”))

Recreate Ds / “refresh table”

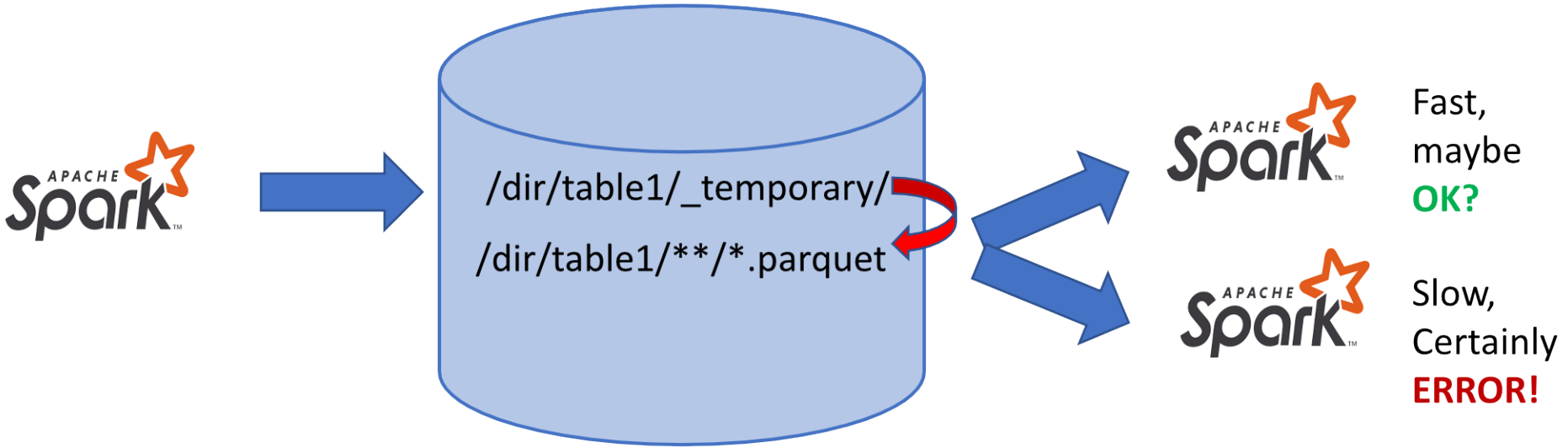
```
26/11/01 15:52:37 ERROR TaskSetManager: Task 0 in stage 4.0 failed 1 times; aborting job
org.apache.spark.SparkException: [FAILED_READ_FILE.FILE_NOT_EXIST] Encountered error while reading file file:///c:/data/
hello-world-dataset/part-00000-3c83366a-4a9f-4c09-8754-907af7f9607a-c000.snappy.parquet. File does not exist. It is poss
ible the underlying files have been updated.
You can explicitly invalidate the cache in Spark by running 'REFRESH TABLE tableName' command in SQL or by recreating th
e Dataset/DataFrame involved. SQLSTATE: KD001
  at org.apache.spark.sql.errors.QueryExecutionErrors$.fileNotExistError(QueryExecutionErrors.scala:902)
  at org.apache.spark.sql.execution.datasources.v2.FileDataSourceV2$.attachFilePath(FileDataSourceV2.scala:140)
```

Spark says :

If it is a dataset ⇒ you need to re-create it (from applicative code)

If it is a SQL over a Table ⇒ call “Refresh TABLE tableName” (spark will re-scan the files)

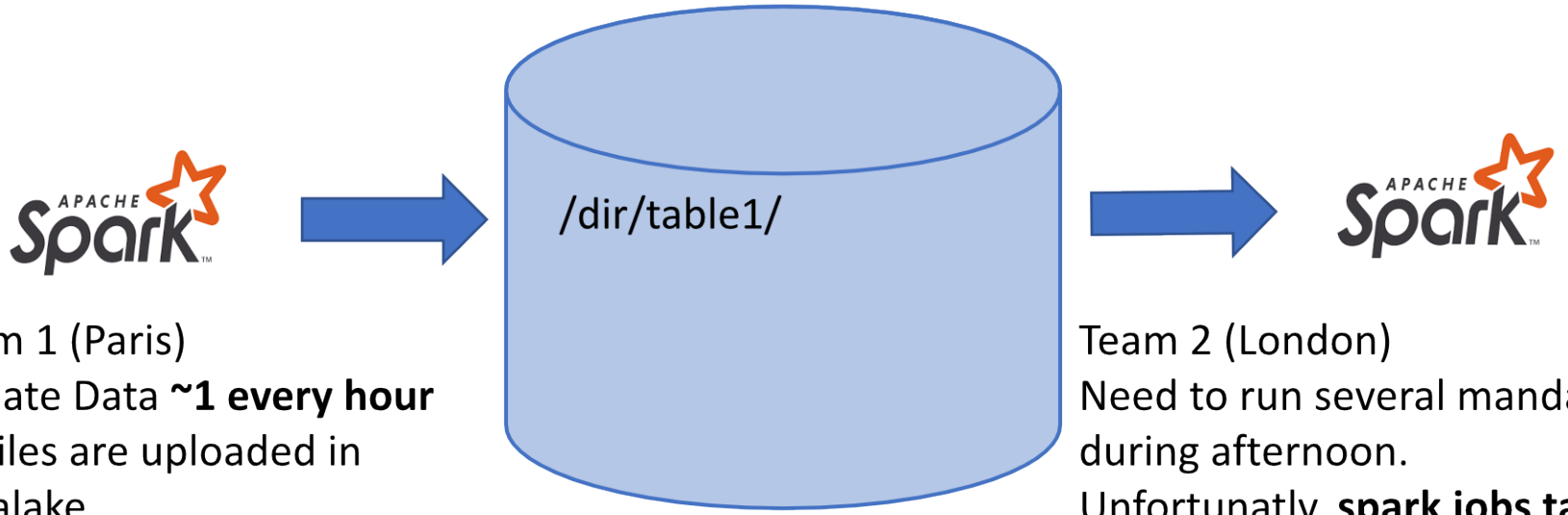
Concurrent Read - Write … DataLake NightMare



Write
OK to _temporary/**
(supposed to be exclusive for 1 “owner”)
On “commit” ⇒ problems occur for other readers

Read
… OK as long as “not modified”
then
… ERROR “File Not Found”

DataLake - FinOps NightMare
Return On Real Experience



Team 1 (Paris)
Update Data **~1 every hour** as Files are uploaded in Datalake

Team 2 (London)
Need to run several mandatory reports during afternoon.
Unfortunatly, **spark jobs take >= 1h**
… Support team relaunch x3, x4 every day

Team noticed that assigning bigger clusters “reduce the occurrence of errors”
⇒ CSV reports now cost >= 50 000 EUR/month
Even after changing strategy (immutable append versions), Team 2 is EURs addict.. Now 100k/

Insert INTO / Insert (Overwrite)

INSERT INTO table values(..) **= mode(“append”)**

=> save to new file(s)

preserve existing ones

INSERT OVERWRITE table .. **= mode(“overwrite”)**

=> reload all files

+ save all to new files

+ delete old files

Sql> Update? DML

by default Spark 3.x does NOT support UPDATE
(nor UPSERT, MERGE)

Only with extensions of « DeltaLake », « Iceberg », ..





Do you really "UPDATE" data ?
… better add Event/Version in new Partition

Remarks :

In general **NO**

Better to **insert a new version** of the object

example: Financial Transactions
... you never "UNDO" the history of a payment,
but you can reimburse, by new payment in opposite direction

Databases actually work like this
ex: Oracle, PostgreSQL have "transaction logs" and multi versions
you can read "as of" in the history

Spark> Update?
read().map().write()

```
spark
    .read().format(« PARQUET »).load(« /data/table1 »)

    .map( x -> {   …transform row to ‘update’ values; return newRow } )

    .write().format(« PARQUET »).mode(SaveMode.Overwrite).save(« /data/table2 »)
```

Full Scan ALL files
Load ALL
in-memory

Process ALL
In-memory

Delete ALL files
+ save ALL
in-memory

Outline

- From Hadoop path + Parquet Files to Table
- Datalake Architectures
  From Hive MetaStore to Iceberg Rest Catalog
- DDL
- DML - read/write Concurrency problems
- Partitioning
- Partition Pruning Optim
- Iceberg Versionning

Smallest Granularity of “ACID”
INSERT / UPDATE / DELETE

INSERT		Add new File(s)
UPDATE DELETE		Delete DIR */* Files + re-add new File(s)

Writing Files (append / overwrite)

Write a single ROW → in 1 new **File**

HDFS hates Small Files
(Too many files) !!

Write from shuffled RDD
(several executors) → in 200 **Files**
by default
spark.sql.shuffle.partitions=200 !!

Overwrite some files,
and no touch others → Possible only by **partition**

Legacy Update Scope

Table Scope for update (~Transactional)



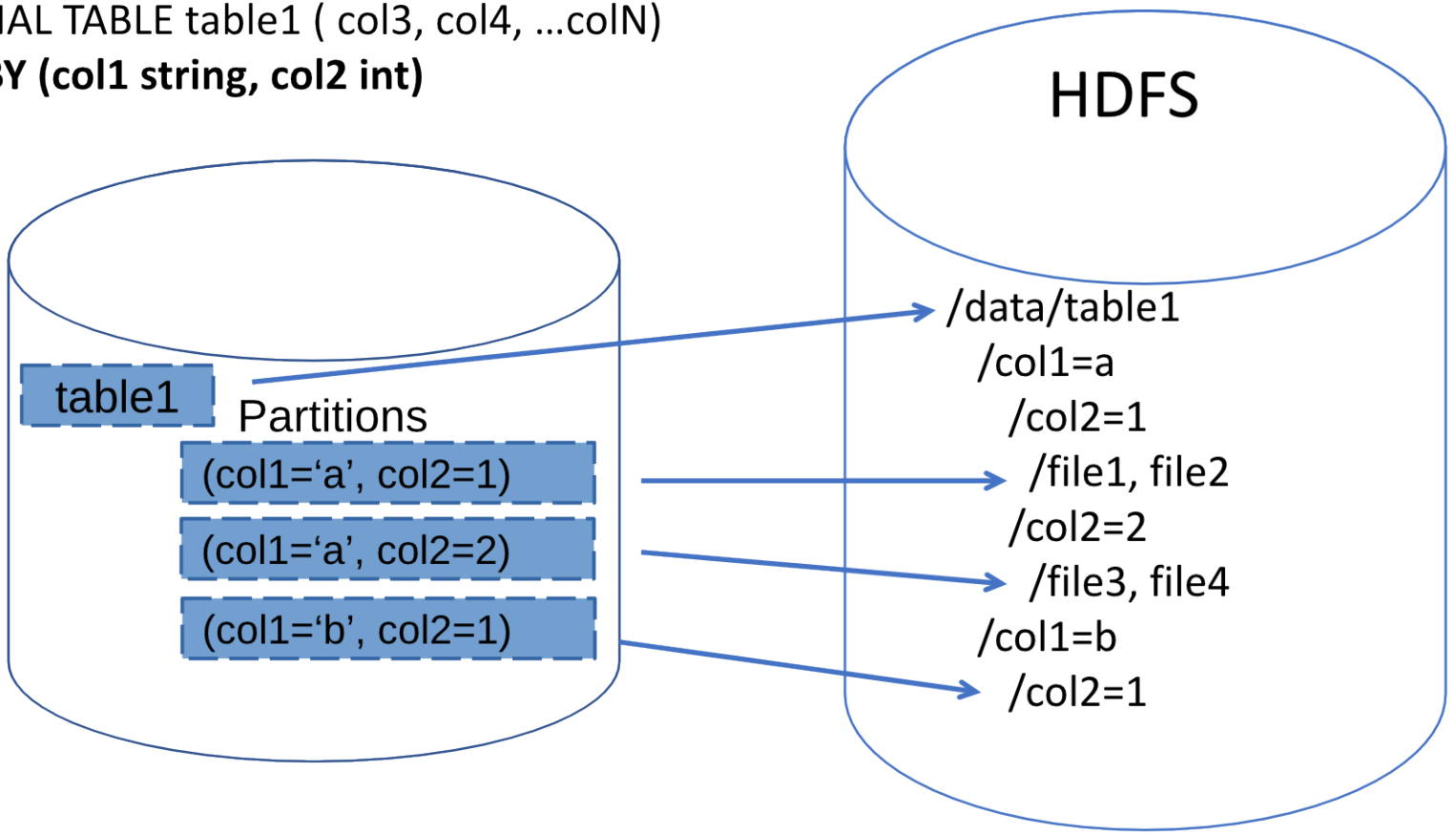
Hive Partition



Hadoop Directory

PARTITIONED BY (col1, col2)

CREATE EXTERNAL TABLE table1 (col3, col4, …colN)
PARTITIONED BY (col1 string, col2 int)



Alter table ADD PARTITION / MSCK REPAIR TABLE

Need EXPLICIT add !!
Otherwise dir/files not scanned => 0 result

Sql>
ALTER TABLE .. **ADD PARTITION** (col1='a', col2=1);
… Or
MSCK REPAIR TABLE ..; -- (inefficient rescan all)

Discover.partitions ??
… False good idea

ALTER TABLE … SET TBLPROPERTIES ('discover.partitions' = 'true')

hive-site.xml
  metastore.partition.management.task.frequency=600

  … => INEFFICIENT : Polling metastore thread every 10mn to scan HDFS, and alter
  + Spark still using explicit partitions

What if you have Petabytes, with millions of dirs?

Daily Batch =>
Partition Dir /date=yyyy-MM-dd

/data/table1

/date=2021-12-25



Partition for today

batch today



relaunch Failed batch ??



/date=2021-12-24/



Already computed
from yesterday's batch
(do not update)

/date=2021-12-23/



…older history

Immutable history

More Complex Daily Batches

/data/table1

/date=2021-12-25

/scope=x/*.*

/scope=y/*.*



sub-partitions
for N batches scope

batch today scope=x



batch today scope=y



/date=2021-12-24/



Already computed
from yesterday's batch
(do not update)

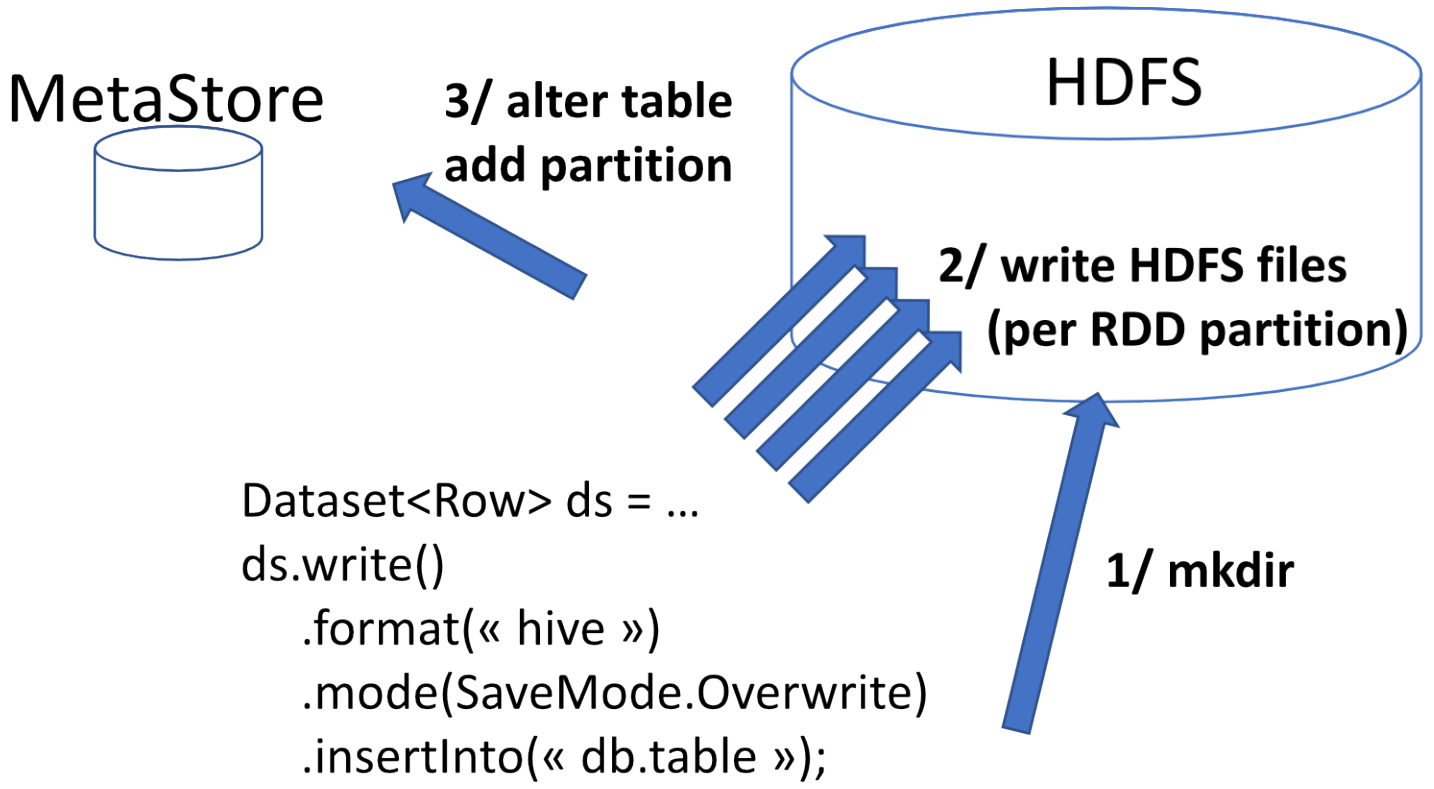
/date=2021-12-23/



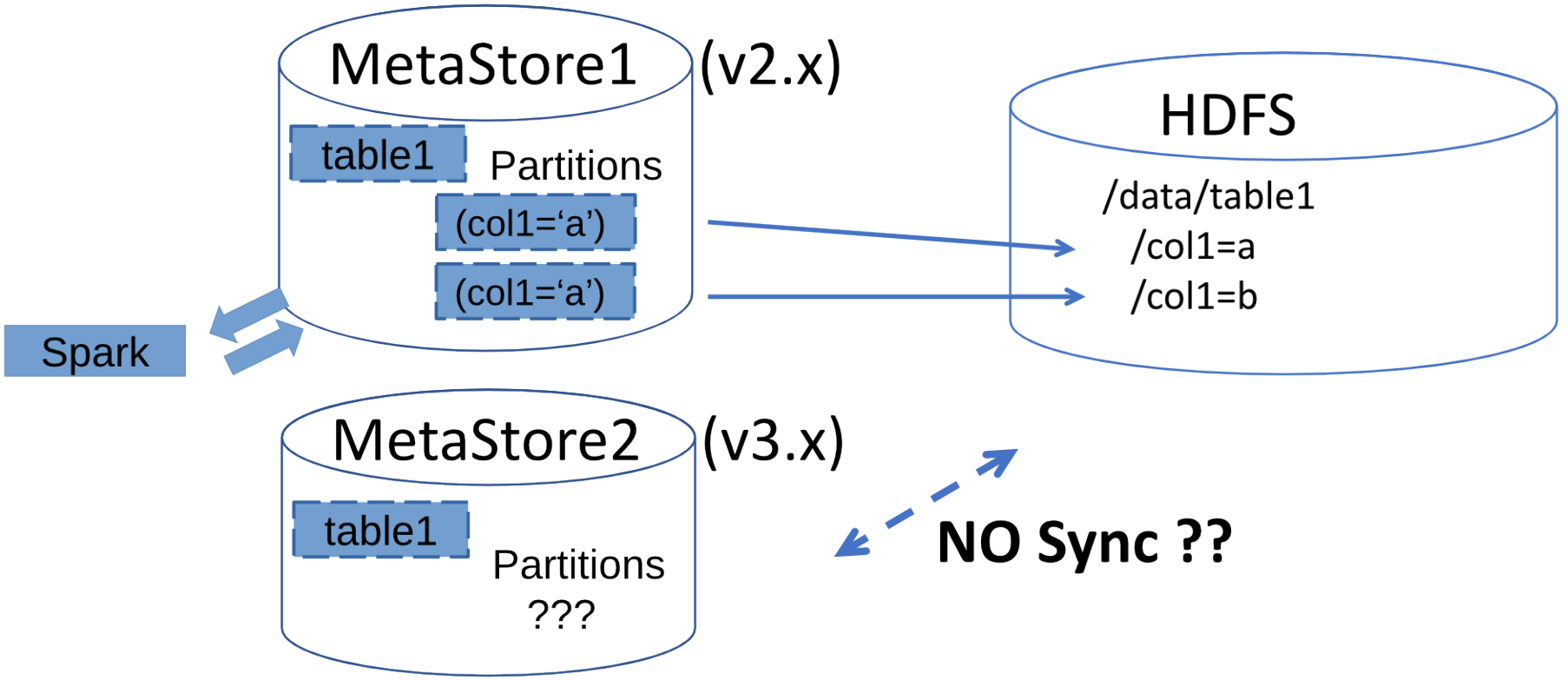
…older history

Immutable history

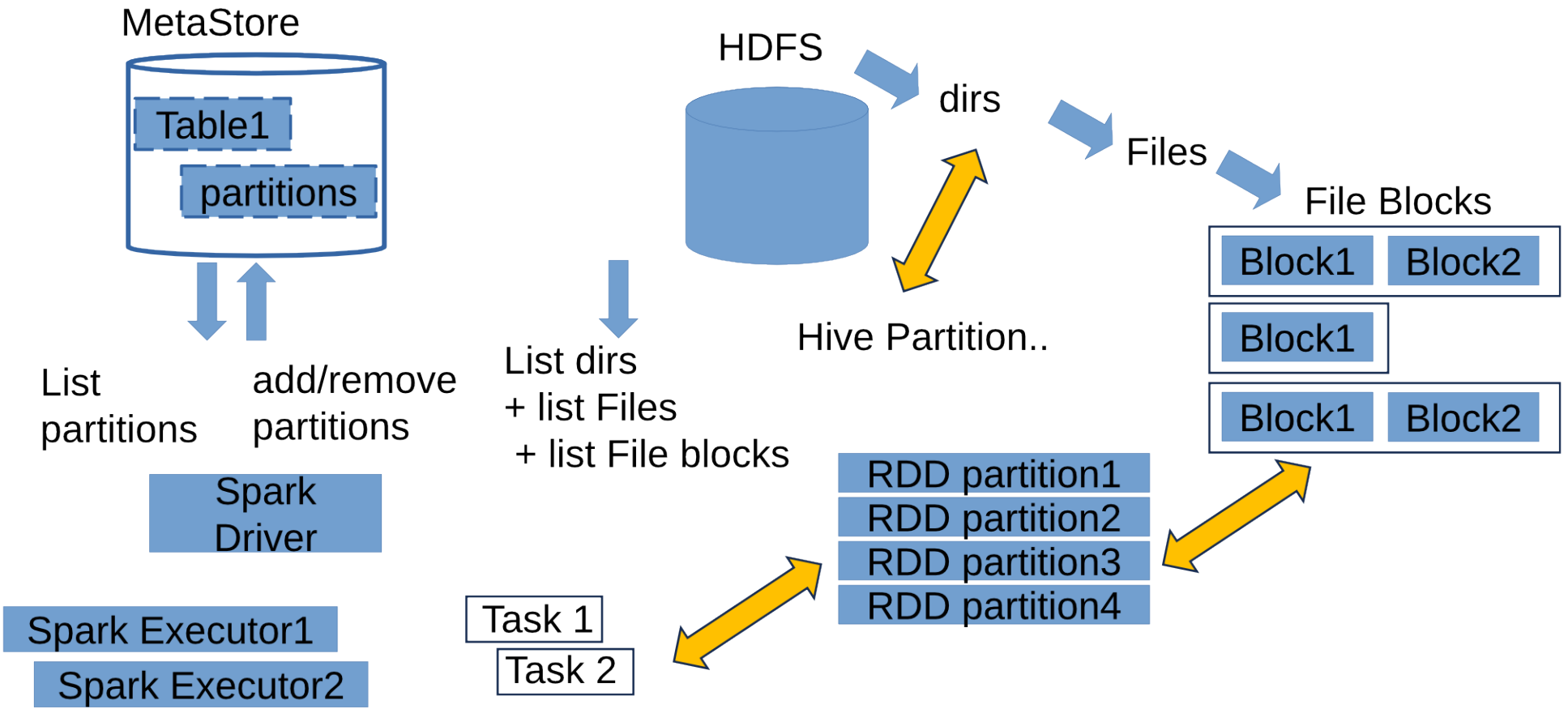
Spark .save()
=> mkdir + write Files + add partition



Synchronize HDFS with several MetaStores?

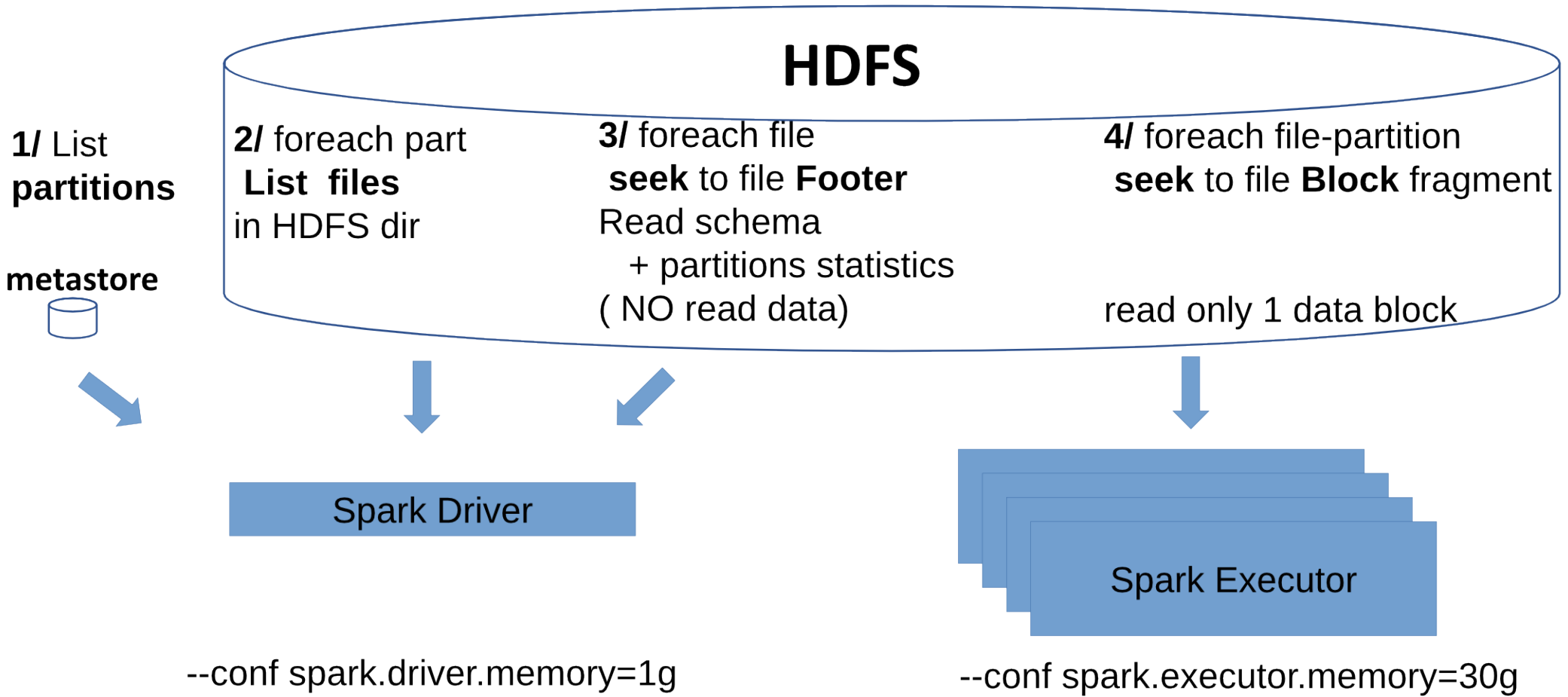


Spark RDD Partitions / MetaStore Partitions



Assign 1 partition to 1 executor = 1 Task (= 1 File Split ~ 1 Block)

Spark RDD Partitions
= MetaStore Partition * Files * Blocks



Outline

- From Hadoop path + Parquet Files to Table
- Datalake Architectures
  From Hive MetaStore to Iceberg Rest Catalog
- DDL
- DML - read/write Concurrency problems
- Partitioning
- Partition Pruning Optim
- Iceberg Versionning

Optim: Partitions Pruning

Sql> select … from db.student where promo=2020 and …

Condition on partitioned column



Scan only files in

/data/student/promo=2020/**

Skip others

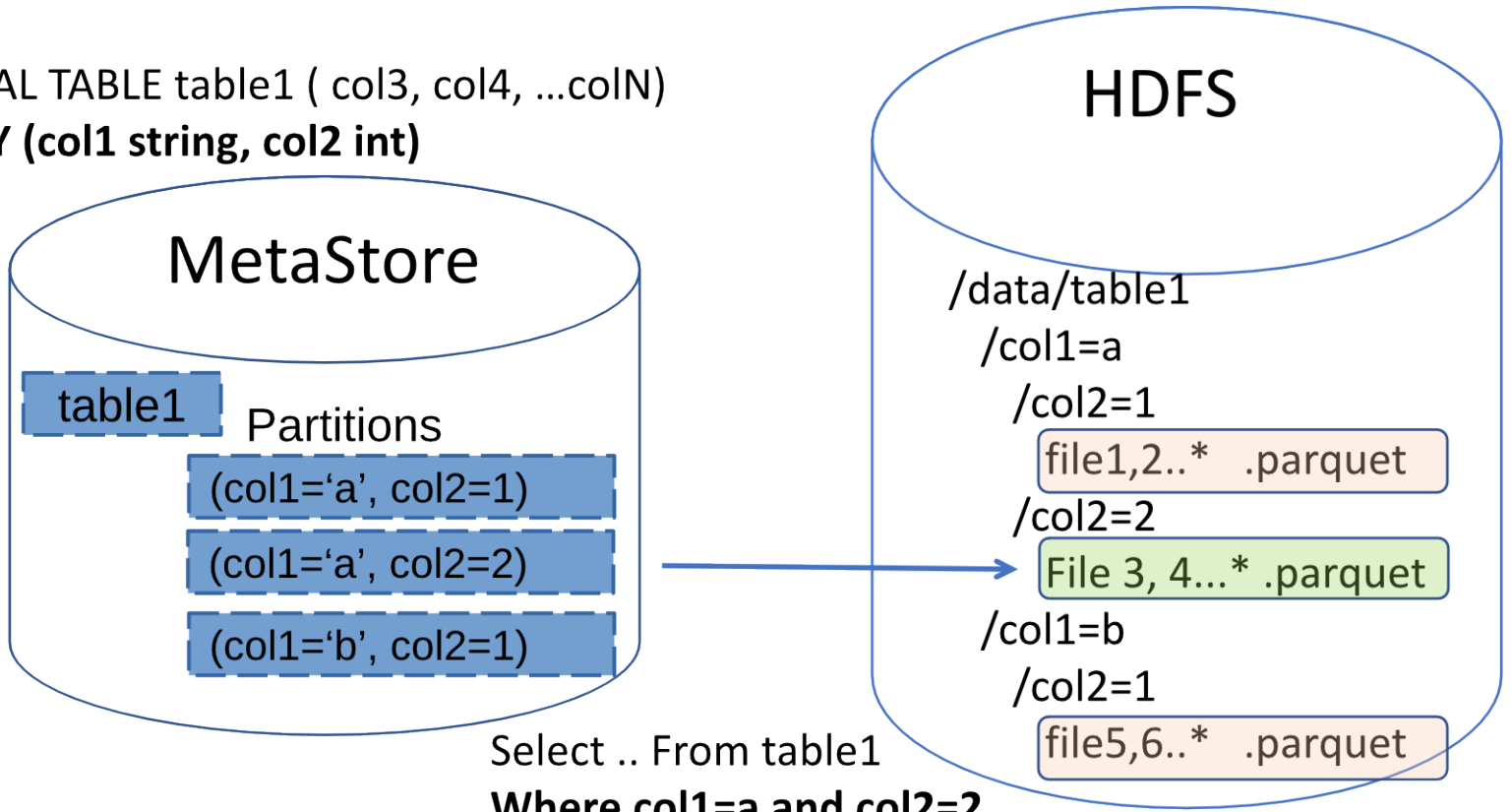
/data/student/promo=2019/

/data/student/promo=2018/

…

Partitions Pruning Optim

CREATE EXTERNAL TABLE table1 (col3, col4, …colN)
PARTITIONED BY (col1 string, col2 int)



Select .. From table1
Where col1=a and col2=2
-- partitioned columns

Partition: what for ?

NOT for searching faster !! (Not-only)

(worse than parquet Predicate-Push-Down)

Granularity of Save mode Overwrite

… adapt to your batch scope

DO NOT define more than 2 partition levels

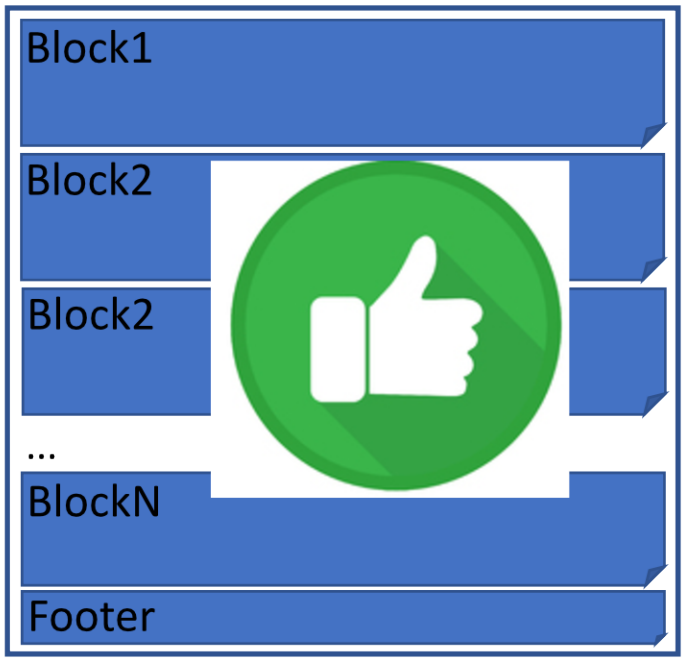
Performances

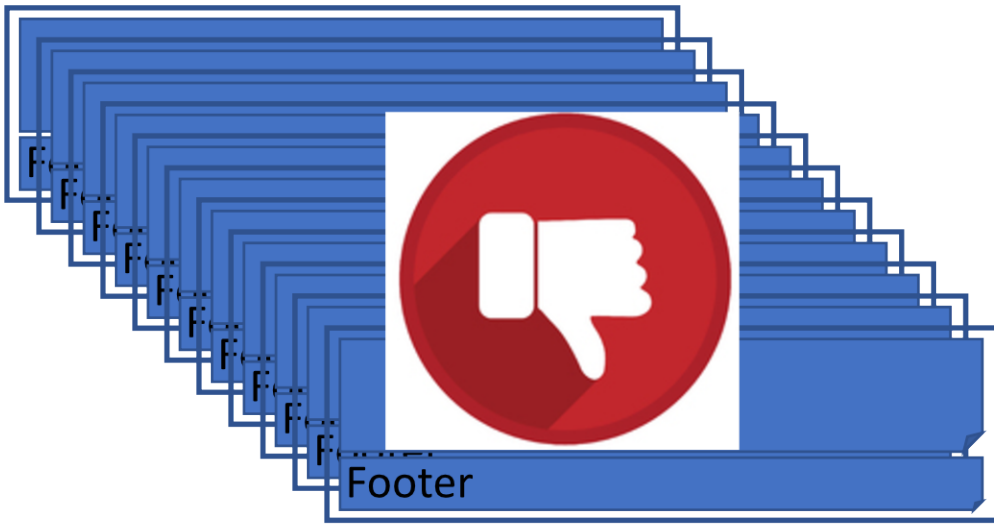
File Blocks >> MetaStore + HDFS Dir + Files

Better to
have 1 Huge HDFS file
(several GB)

than

Too MANY
Too Small files
(few 128+1 MB)





Typical Partition / Files Volumes

For daily batch

1 partition per day … 5 year of data = ~1500 partitions OK

1 file per partition … OK, even if strange to have 1 file per directory

(maybe 2,3 files per partition … if no fit in spark executor mem)

File may be >= several Giga bytes …. OK great

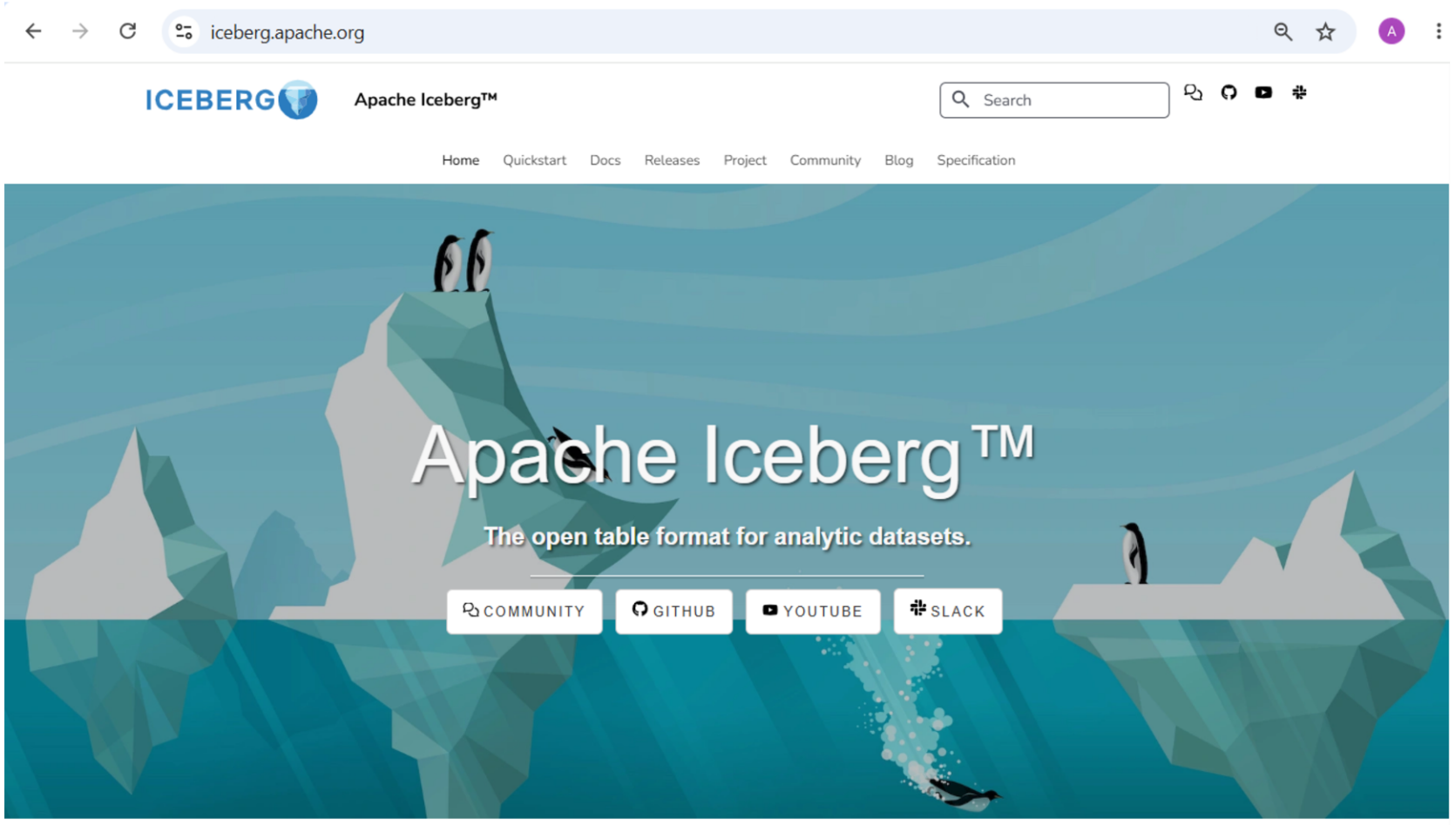
File parquet.block.size = 16M, 32M (? overwrite default 128M)

compromise:
Smaller => more dictionary encoding,
better PPD, maybe less compression
Bigger => less partitions

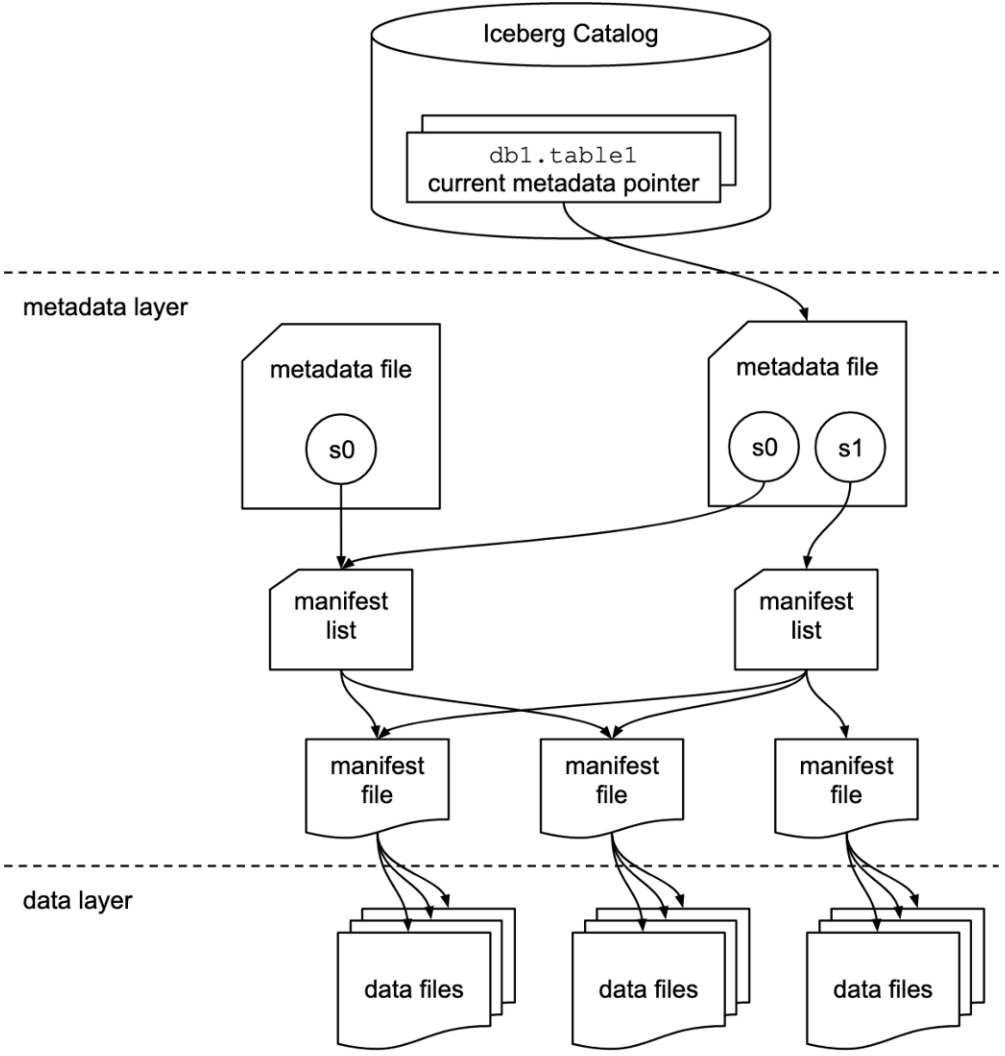
Outline

- From Hadoop path + Parquet Files to Table
- Datalake Architectures
  From Hive MetaStore to Iceberg Rest Catalog
- DDL
- DML - read/write Concurrency problems
- Partitioning
- Partition Pruning Optim
- Iceberg Versionning

https://iceberg.apache.org



https://iceberg.apache.org/spec/#goals

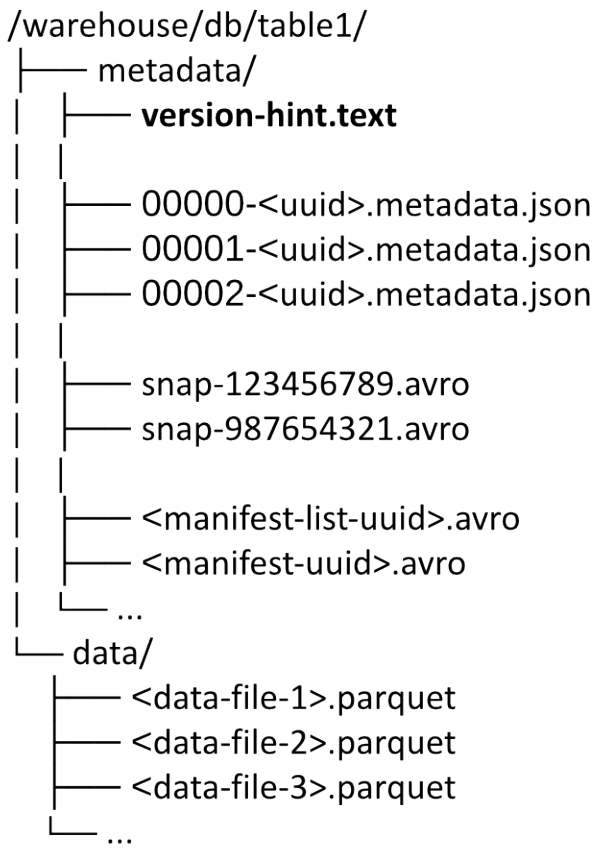


Iceberg ... Transaction Log Files

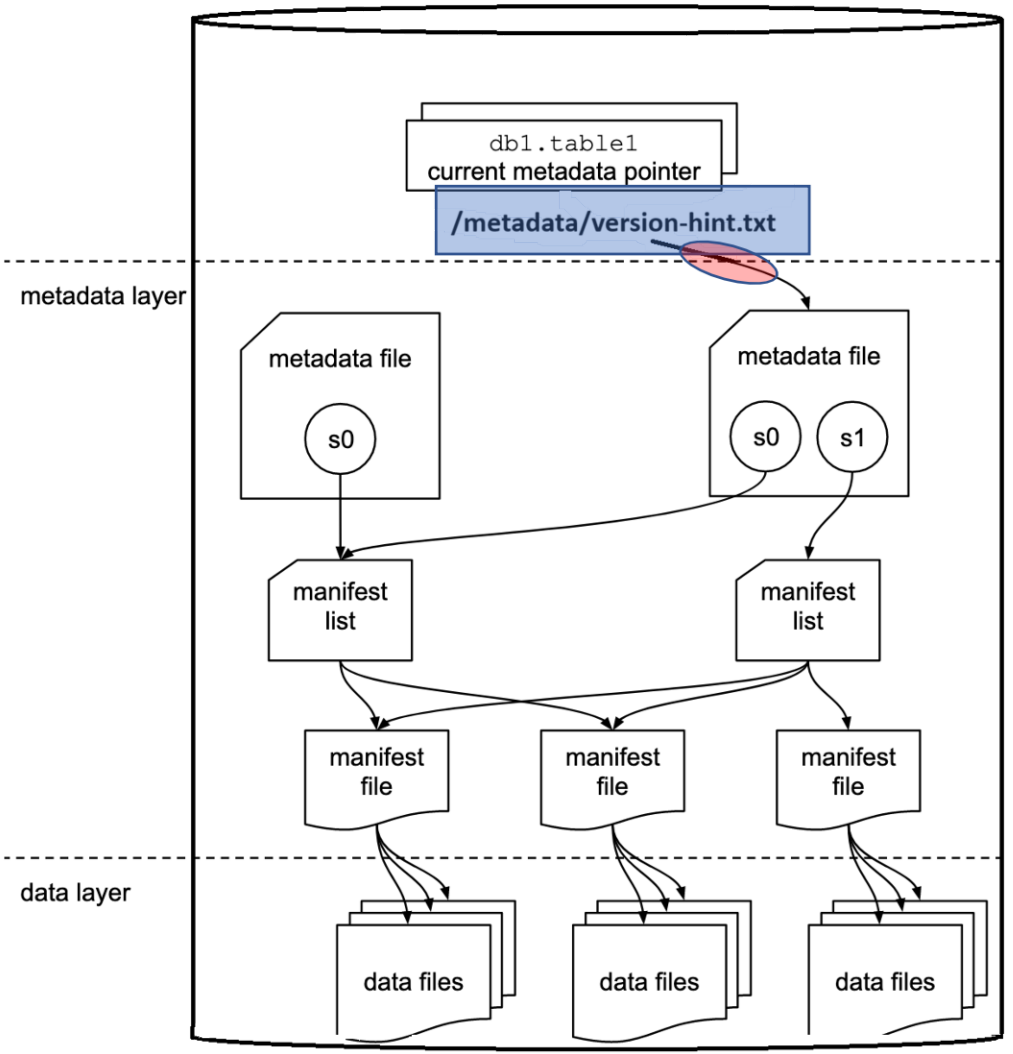
Iceberg = use its own custom MetaStore

not based on listing all Files in a Directory (like in Hive)
BUT by keeping a transactionLog (in json files)

Iceberg over Hadoop Storage



Alternative Architecture 1: Iceberg over HADOOP



{table}/
/metadata
/version-hint.text

Commiting (atomically?)
= changing this version file

ACID on HDFS / S3 / Abfs ??
How to switch Version

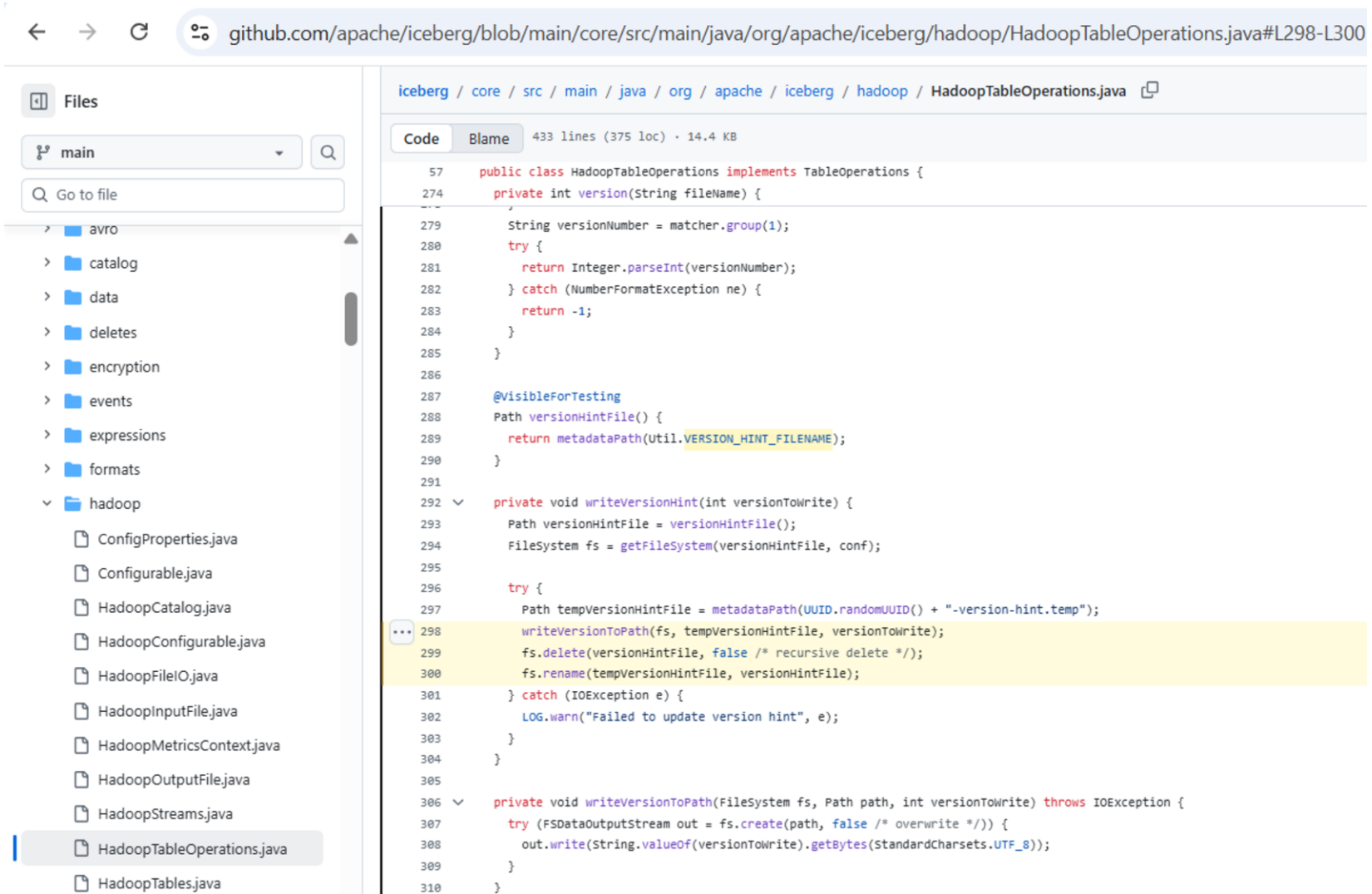
To be ACID,
Iceberg need to Atomically change file “/version.txt”
updating text content (example) “v42” to “v43”

On “Classical” FileSystem (linux) : OK
    Lock, Truncate, Write, FLUSH, unlock

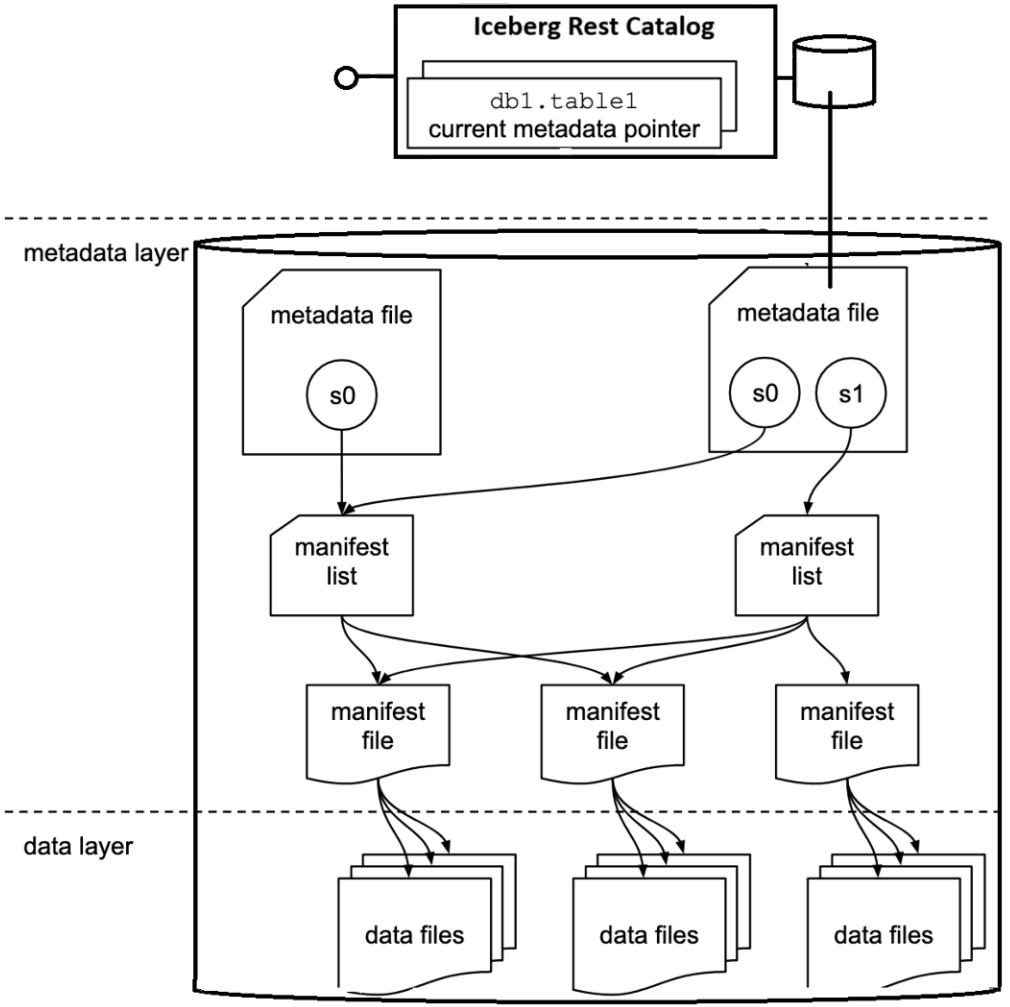
BUT on HDFS / S3 / Abfs storage:
    Write to “/version-new-{UUID}.txt”
    DELETE “/version.txt”
    RENAME “/version-new-{UUID}.txt” → “/version.txt”

… This is NOT Atomic !

Github … hadoop/HadoopTableOperations.java#L298-L300



Iceberg using IRC - atomic (ex Postgres DB)



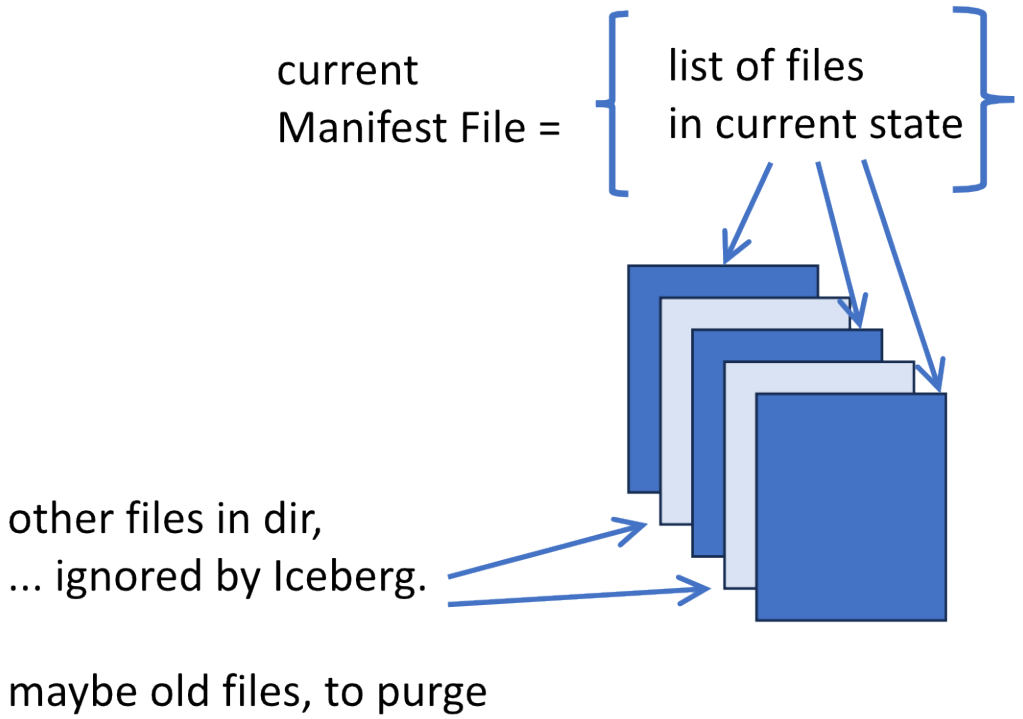
Iceberg =
Json/Avro + Parquet Files

JSON + Avro File for storing Metadata (transaction logs)

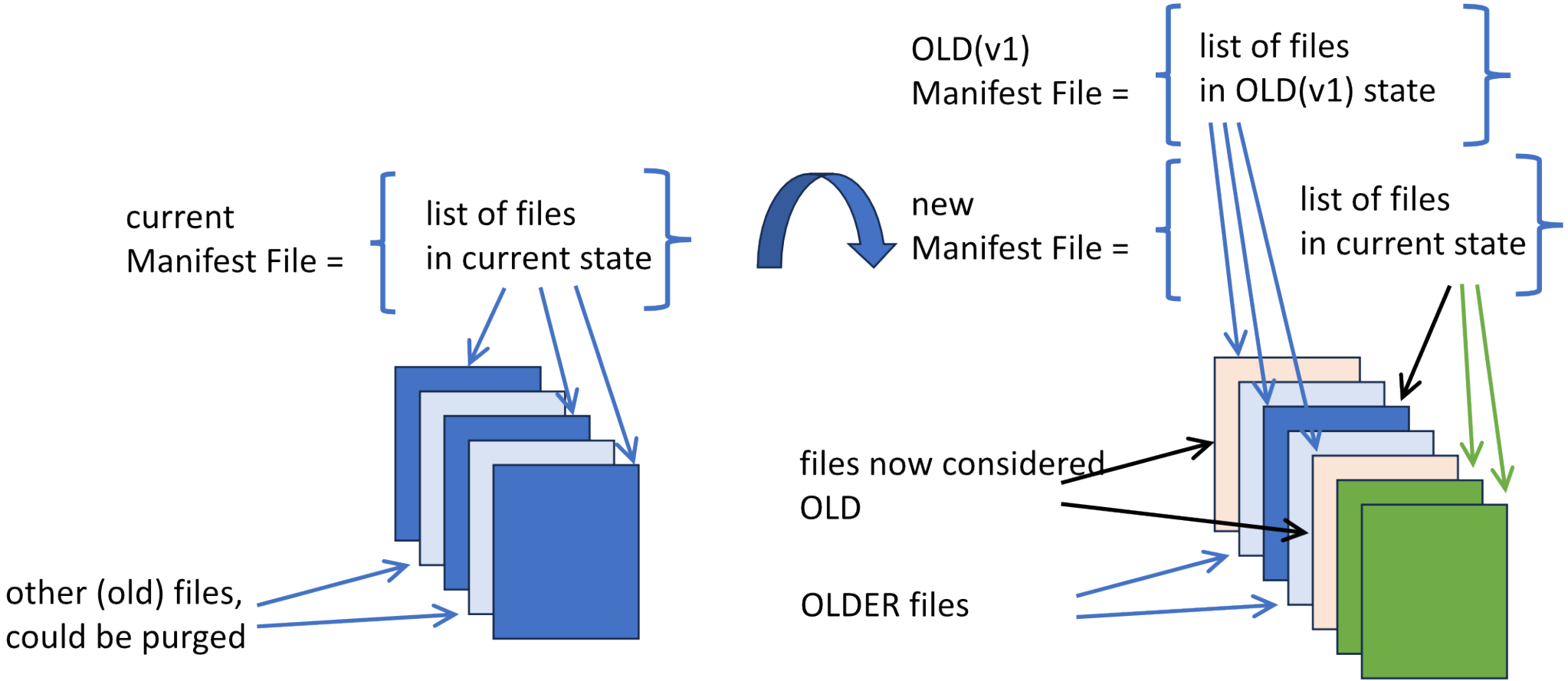
Parquet Files for storing Data (like plain old spark + Hive did well)

Transactions no more handled by "directory listing"
No need "hive partition"
Only need Hive "DDL"

Iceberg Manifest File State

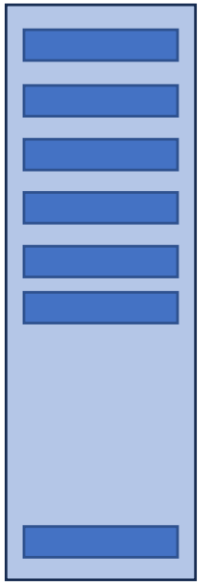


Iceberg Update = Transaction to disable (old) Files, and add new Files



Update p rows (p << N) : Duplicate All
Copy On Write

Previous file
has N(millions) rows



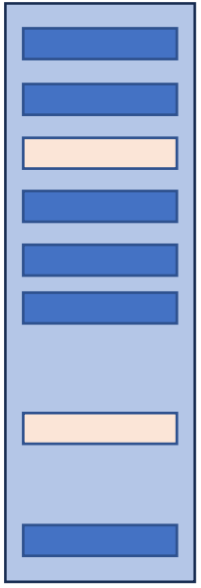
need to update/delete
only “p” rows (p << N):
at index 3, 42, ..

Row 3

Row 42

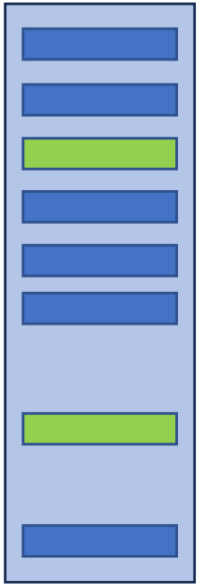


File v1
(keep all)

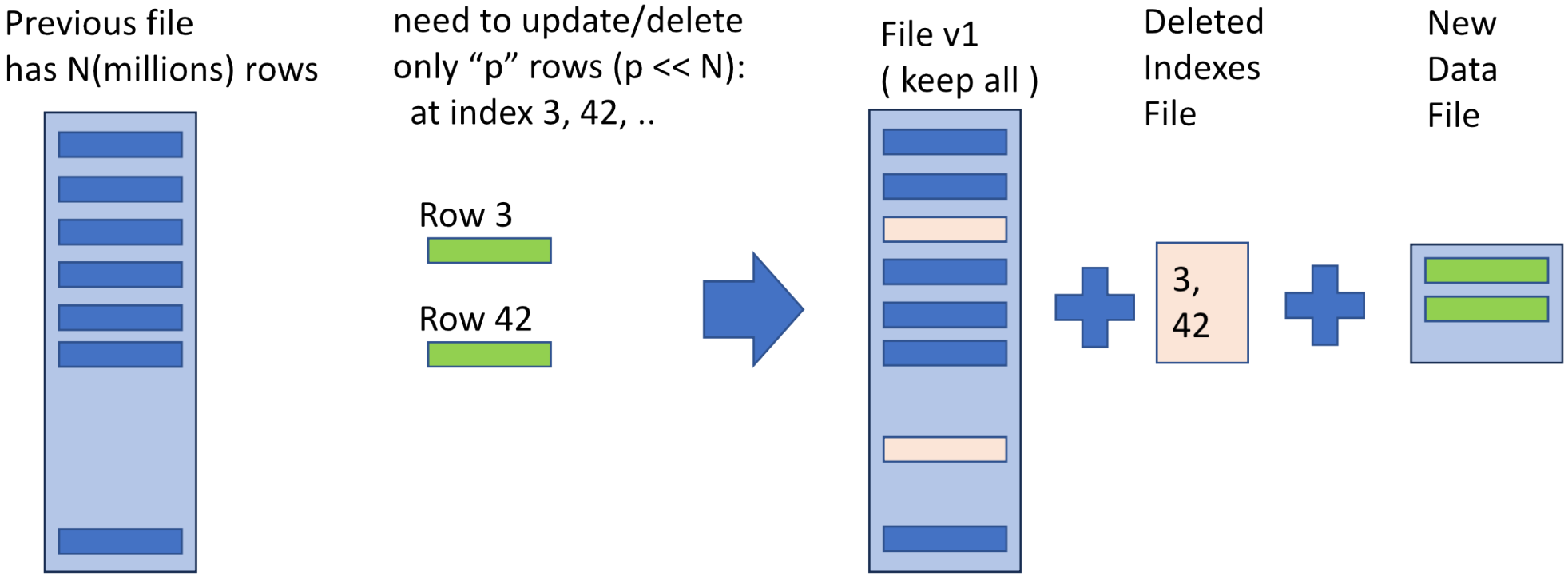




File v2
(prev rows + new rows)



Iceberg “Disable Rows” by Indexes Merge On Read



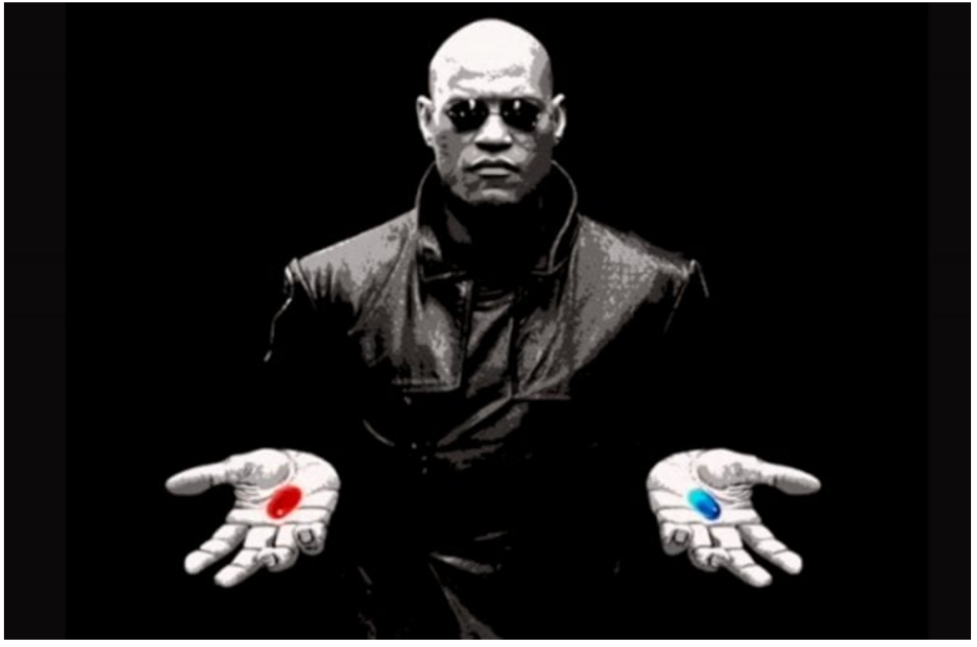
Files Number Explosion Problem

Both “Copy On Write” & “Merge On Read”
produce **several new files on each commit**

known as “**Files Number Explosion**” problem

Impact performances, need purge

Copy On Write / Merge On Read



Iceberg “Copy On Write” to Update 1 row of a File => duplicate 100000 rows in new file !

Data Storage Duplication Size
+ “**Files Number Explosion**” Problems ...

Need **Purge** mechanism
example: retention to 7 days of history

Iceberg “Merge On Read”

To read a table …

read thousand of useless (deprecated) rows,
recompute to retain only latest ones

Take Away

Apache Parquet is the DE-FACTO standard for File storage

In Legacy Datalake = using Hive MetaStore, with Partitions=sub-Directories

Updating = delete + recreate files in Directories ...
But Listing Files in Dir is SLOW

Apache Iceberg solve the Listing + Versionning

Questions ?